EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Week 7: Retrieval-Augmented Generation (RAG) and Research Agents

Answering with sources and choosing further searches

Instructor: Arash Saifhashemi

EE 599 SESSION 07

RAG and research agents

How can an agent find information and produce an answer whose **claims we can check**?

This lecture connects **document retrieval**, answering with sources, and research that uses several searches.

RAG and research agents: scope and purpose

How can an agent answer from sources and choose what to investigate next?

RAG	Retrieval-augmented generation: find source text and give it to a language model when it writes an answer.
Chunk	A short piece of text stored and retrieved as one unit.
Research agent	An AI agent that reviews its findings, checks whether it can answer the question, and chooses another search or writes an answer.

A claim is a statement we can check. A citation identifies its source.

Lewis et al. (2020)

Lecture roadmap and learning goals

Each section answers the question left open by the previous one.

1	**RAG foundations**	Connect a question, source chunks, and a cited answer.
2	**Prepare the sources**	Keep context and source locations with each chunk.
3	**Find useful chunks**	Represent text as vectors (lists of numbers) and compare them.
4	**Build a RAG answer**	Select evidence, assemble the input, and generate.
5	**Check answer quality**	Check claims, citations, and missing information.
6	**Choose another search**	Use earlier findings to decide what to look for next.
7	**Choose an approach**	Compare alternatives and diagnose failures.

Four papers we will discuss

Paper	Teaching purpose
DPR (Dense Passage Retrieval) Karpukhin et al. (2020)	Retrieve matching text using vectors
RAG · Lewis et al. (2020)	Combine retrieved text with generation
ALCE · Gao et al. (2023)	Evaluate answers and citation support
Self-RAG · Asai et al. (2024)	Learn when to retrieve and check an answer

Concepts come before the paper mechanisms that use them.

DPR uses “passage” for a retrieved text chunk.

RAG: answering with sources

How can a model answer from a source it did not learn during training?

Start with a concrete question, the source text, and the answer we want to check.

Then identify the retrieval and generation steps that connect them.

Running example: a first visit to Los Angeles

Help friends plan a Saturday visit to these three landmarks.



Griffith Observatory

Serouj · Public domain



Hollywood Boulevard

Benoît Prieur · CC0



Santa Monica Pier

Bernard Spragg. NZ · CC0

First: hours and parking. Later: a no-car itinerary.

Two pieces of information we need: hours and parking

Our question:

- Is the Griffith Observatory building open at 7 p.m. on Saturday?
- where can we park?

A **chunk** is a short piece of text from a document.

C1 (Chunk 1): Building hours

Saturday building hours:
10 a.m. to 10 p.m.

Hours: Yes, 7 p.m. is within the listed Saturday building hours [C1].

C2 (Chunk 2): Parking locations

Paid parking: Observatory lot and West Observatory Road.

Parking: Paid parking is in the Observatory lot and along West Observatory Road [C2].

We need both chunks to answer both parts of the question.

The brackets identify the chunk supporting each statement.

Original excerpts: Griffith Visit; Getting Here. Captured 21 Oct 2026.

A useful answer connects each claim to a chunk

answer claim

Saturday 7 p.m. is within hours [C1]

[C1]

original chunk

C1: Saturday building hours: 10 a.m. to 10 p.m.

Source: Griffith Visit page, Saturday hours; saved 21 Oct 2026

- **Example answer:**
  - The Saturday building hours include 7 p.m. [C1].
  - Paid spaces are in the Observatory lot and along West Observatory Road [C2].

LA source snapshot: 21 Oct 2026.

Why answers need source evidence

Closed-book answering uses what the model learned during training, without retrieving a source document.

Question: When does the Griffith Observatory building close on Saturday?

Invented model answer

"The building closes at sunset."

Source chunk C1

Saturday building hours:
10 a.m. to 10 p.m.

Source-based answer: The building closes at 10 p.m. on Saturday [C1].

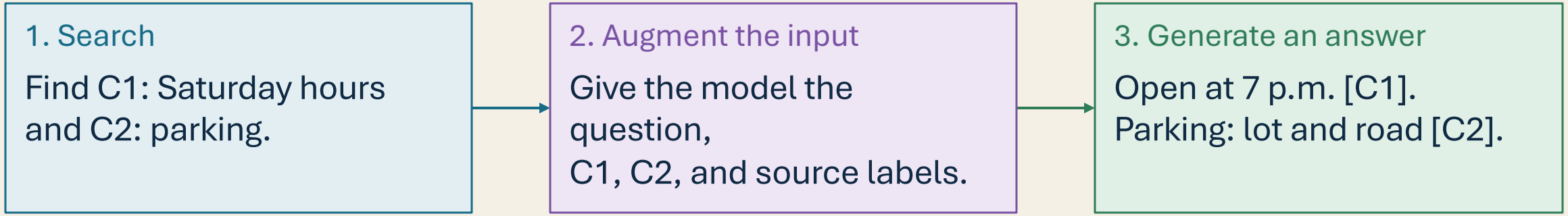
Why retrieve? Training may omit an uploaded guide or changes made after training.

LA source snapshot: 21 Oct 2026. Lewis et al. (2020)

Search is part of RAG. An agent is optional.

Question: Is Griffith Observatory open at 7 p.m. on Saturday, and where can we park?

RAG: retrieval-augmented generation



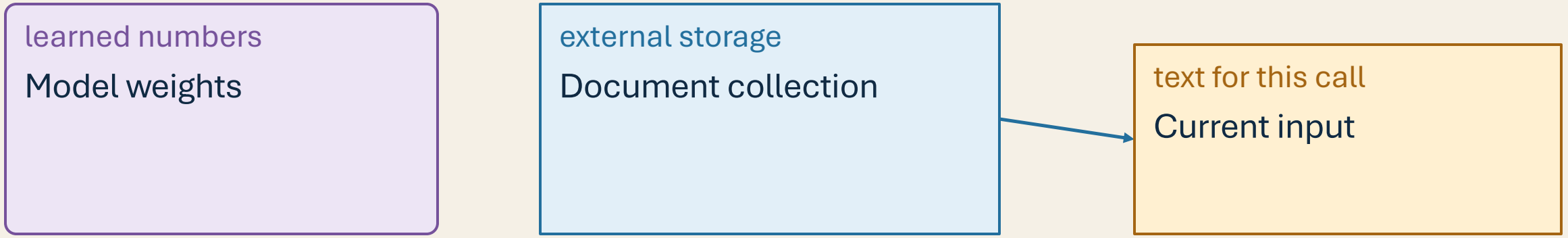
Augment means adding the retrieved source text to the model's input.

Optional research agent

Checks the findings and decides: search again or answer.
If only C1 was found, search for parking next.

Lewis et al. (2020) Asai et al. (2024) Shao et al. (2024)

Where does each kind of knowledge live?



Training changes these numbers.

Retrieval selects from these texts.

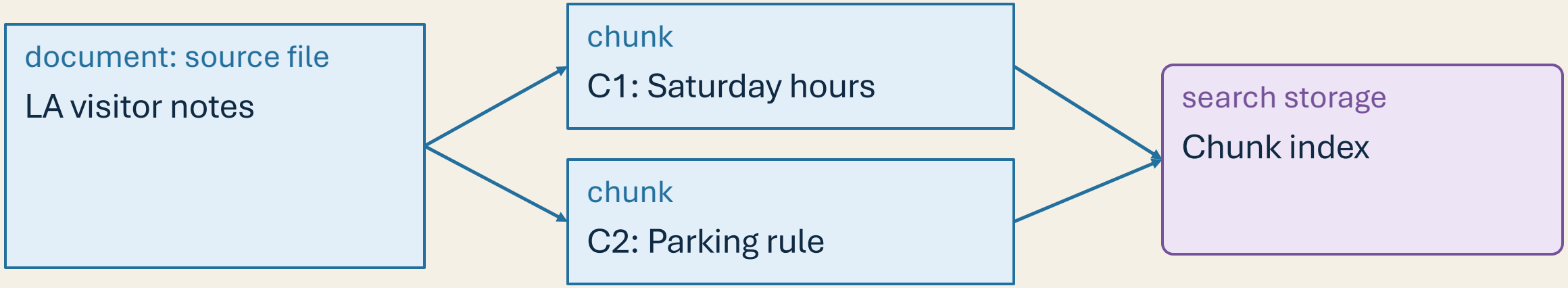
Prepare the document collection

What must each stored chunk contain to remain useful when retrieved alone?

We have a question and source documents. Now we need searchable units that retain their meaning.

Split the text, preserve its context, and keep its source location.

What does a retriever search?



- **Document**: a source file or page, such as the Griffith Observatory Visit page.
- **Chunk**: a portion stored as one retrieval unit. C1 and C2 are examples.
- **Index**: searchable representations and IDs that point to the saved chunks and help locate their text.

LA source snapshot: 21 Oct 2026. Karpukhin et al. (2020)

Keep enough context to interpret a chunk

A retriever may return one chunk without the surrounding document.

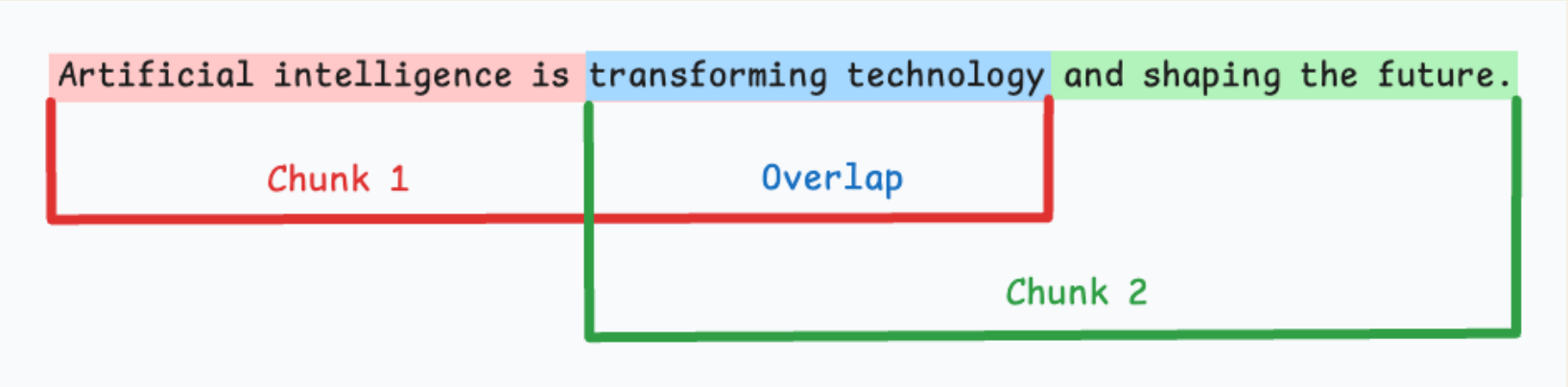
Question: Is the Griffith Observatory building open at 7 p.m. on Saturday?

Missing context	Enough context
Bad C1: 10 a.m. to 10 p.m.	Good C1: Griffith Observatory Saturday building hours: 10 a.m. to 10 p.m.

Answer from Good C1: Yes, 7 p.m. is within the listed Saturday building hours [C1].

Chunking choices: Fixed size

Choose where to split the text, then decide whether adjacent chunks should share text.



Method	Benefit	Cost or risk
Fixed size No overlap	Simple splitting No duplicated text	May separate a rule from its heading
Fixed size With overlap	Keeps some context across adjacent chunks	Repeats text More storage and model input

Microsoft Learn (2026) Karpukhin et al. (2020)

When does chunk overlap help?

Question: How do we pay, and what do we display?

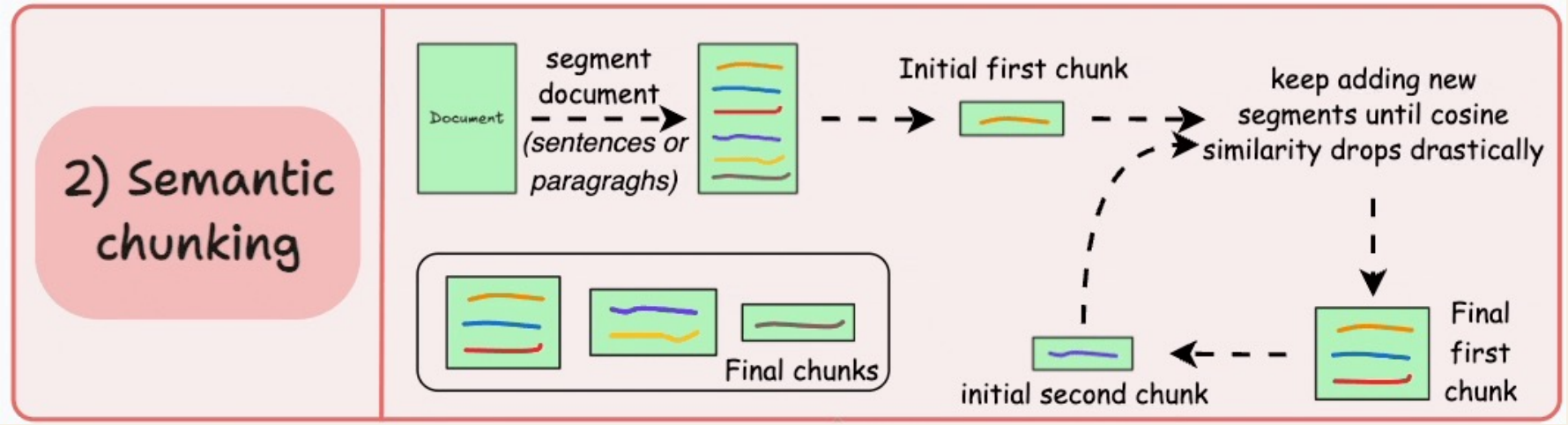
1	Paid parking: Observatory lot and West Observatory Road.
2	Pay stations take credit cards only.
3	Display the receipt on your dashboard.

Split	Chunks	Evidence needed for this answer
No overlap	[1, 2] + [3]	Retrieve both chunks
One-sentence overlap	[1, 2] + [2, 3]	[2, 3] contains both requested facts
Whole section	[1, 2, 3]	One larger chunk contains both facts

Sentence 2 repeats in two chunks: more indexed text and potentially more input text.

LA source snapshot: 21 Oct 2026.

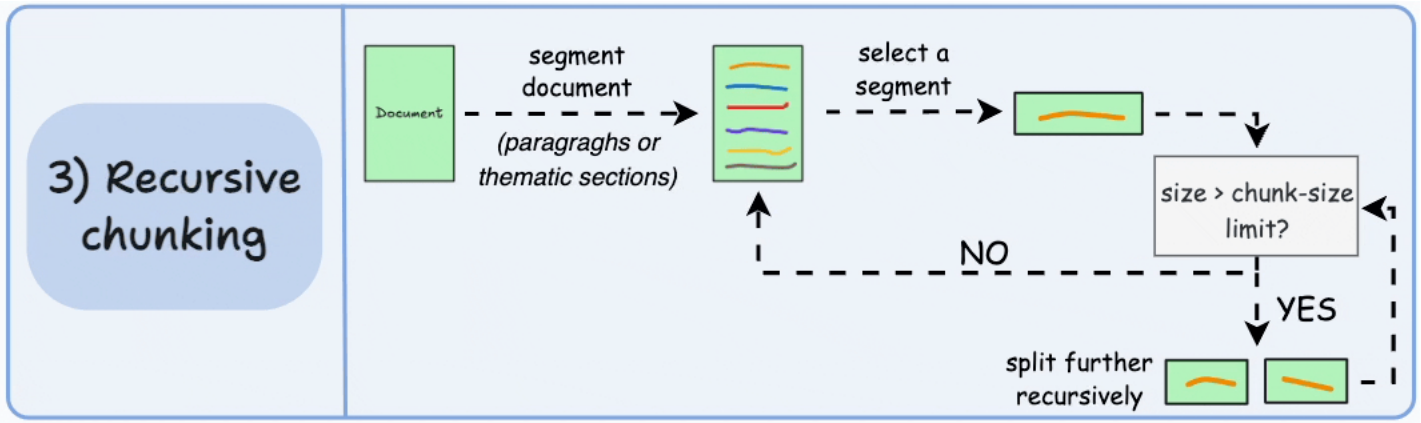
Semantic Chunking



Artificial intelligence is transforming industries by automating processes, enhancing decision-making, and providing insights through data analysis. Machine learning, a subset of AI, enables systems to learn and improve from experience without explicit programming. Deep learning, a branch of machine learning, uses neural networks with multiple layers to model complex patterns in data.

Daily Dose of Data Science: chunking strategies

Recursive Chunking



Paragraph 1

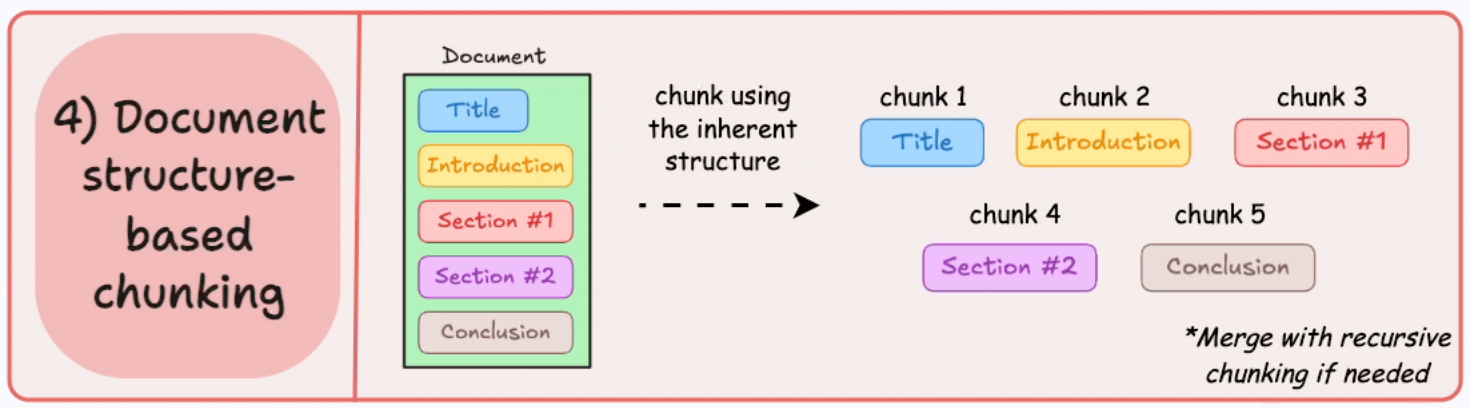
Artificial intelligence is transforming industries by automating processes, enhancing decision-making, and providing insights through data analysis. Machine learning, a subset of AI, enables systems to learn and improve from experience without explicit programming. Deep learning, a branch of machine learning, uses neural networks with multiple layers to model complex patterns in data.

Paragraph 2

AI is also improving natural language processing, enabling applications like chatbots and virtual assistants.

Daily Dose of Data Science: chunking strategies

Document Structure Chunking




```
Title: The Role of Artificial Intelligence in Modern Education
```
Chunk 1

```
Introduction

Artificial intelligence (AI) is reshaping education by providing
personalized learning experiences and automating administrative tasks.
```
Chunk 2

```
Section 1: Personalized Learning

AI enables the customization of educational content to meet individual
student needs, enhancing engagement and comprehension.
```
Chunk 3

```
Section 2: Administrative Automation

From grading to scheduling, AI tools are streamlining administrative
processes, allowing educators to focus more on teaching.
```
Chunk 4

```
Conclusion

The integration of AI in education holds the promise of more efficient
learning environments and improved student outcomes.
```
Chunk 5

Illustration: Daily Dose of Data Science

There can be more!

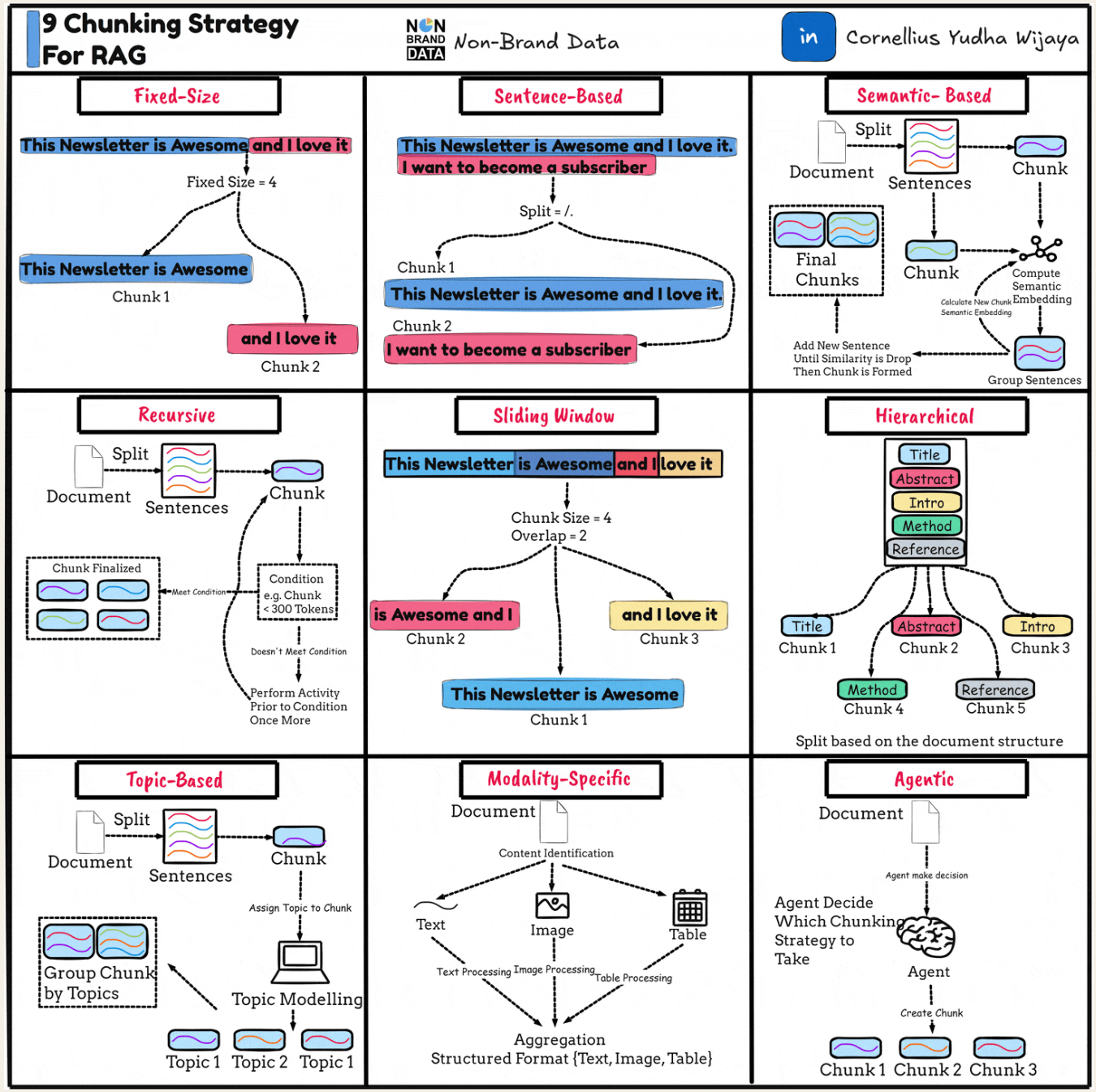
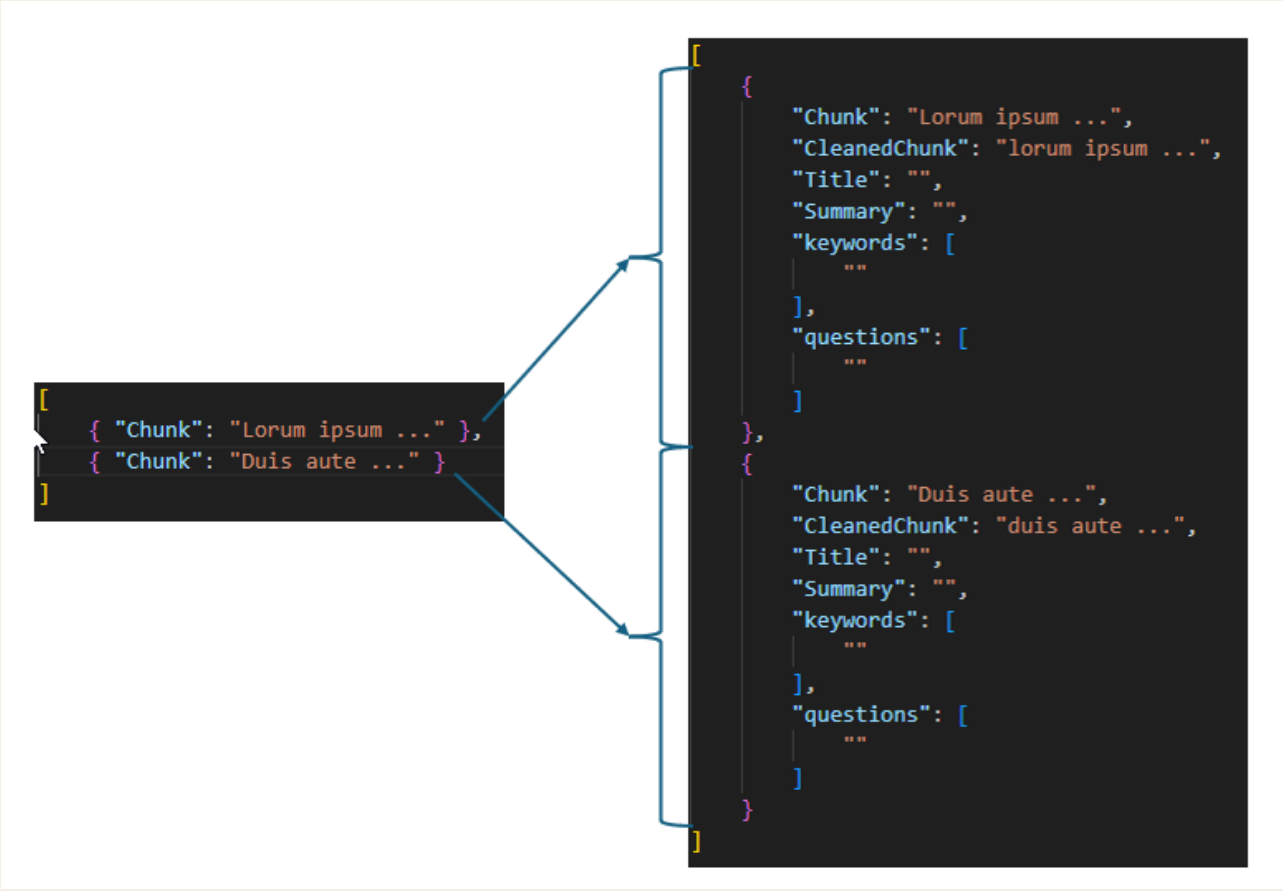


Illustration:
Cornellius Yudha Wijaya
Non-Brand Data

Keep each chunk with its source location

Metadata records where a chunk came from, so its text can be traced back to the source.



Also store the source URL and the section or page number. These fields let us trace a retrieved chunk back to its document.

Represent and retrieve text

How do we find useful chunks when the question uses different words?

The collection now contains readable chunks with source locations.

Represent text as vectors, compare them, and inspect the DPR retrieval method.

Keyword, semantic, and hybrid retrieval

Question at Griffith Observatory: “Where can I leave my car?”

Chunk C2: “Paid parking is available in the Observatory lot and along West Observatory Road.”

Approach	How it searches	Strength and limitation
Keyword **(lexical)**	Looks for shared words. No “**car**” or “**leave**” in C2.	Good for **exact names**. May miss different wording.
Semantic	Estimates **similarity** in meaning. “Leave my car” relates to “parking.”	Can handle **different wording**. May retrieve unhelpful text.
Hybrid	**Combines** keyword and semantic search results.	Uses both matching methods. Needs a rule to combine results.

BM25 ranks word matches using word frequency and document length.
Read each retrieved chunk before using it as evidence.

Karpukhin et al. (2020) Microsoft: hybrid search Griffith: Getting Here

What is a text embedding?

A text embedding is a learned list of numbers that represents a word, sentence, question, or chunk.

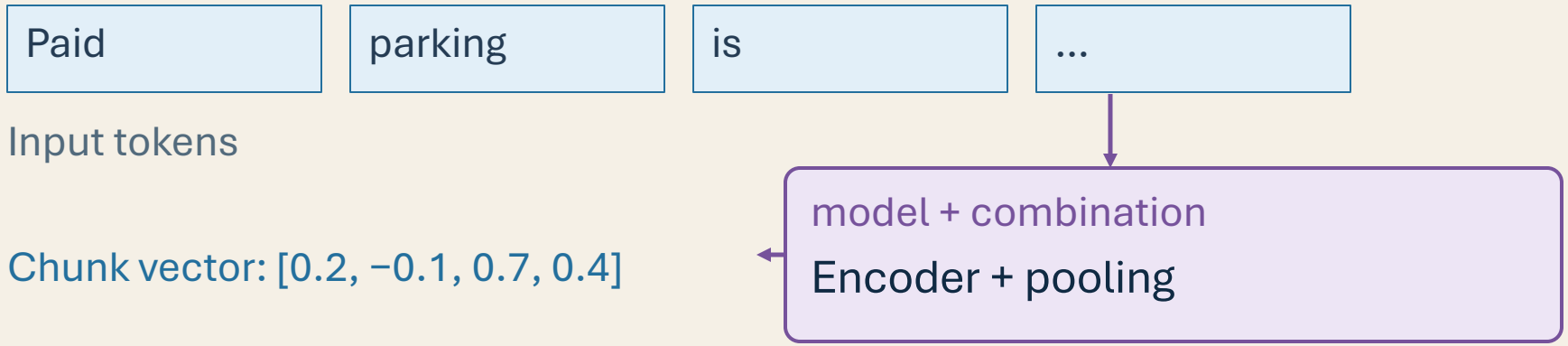
Input text	Example embedding
“Where can I leave the car?”	[0.80, 0.60, 0.10, 0.30]
“Paid parking near the Observatory”	[0.78, 0.62, 0.12, 0.28]

Training can make **related inputs** receive **similar vectors**, even when their words differ.

Karpukhin et al. (2020)

From text to embedding

Text excerpt: “Paid parking is available in the Observatory lot.”



- **Encoder:** Converts input into representations. Tokens are pieces processed by the model; their representations reflect context.
- **Pooling:** Combine token representations into one vector, for example by averaging.

Karpukhin et al. (2020)

Training text embeddings

Training uses question–chunk pairs to teach the encoder which text should match.

Question	Training chunk	Desired ranking
Where can I leave the **car** near Griffith?	C2: **paid-parking** rule Positive example	Rank **higher**
The same parking question	C3: Pier location Negative example	Rank **lower**

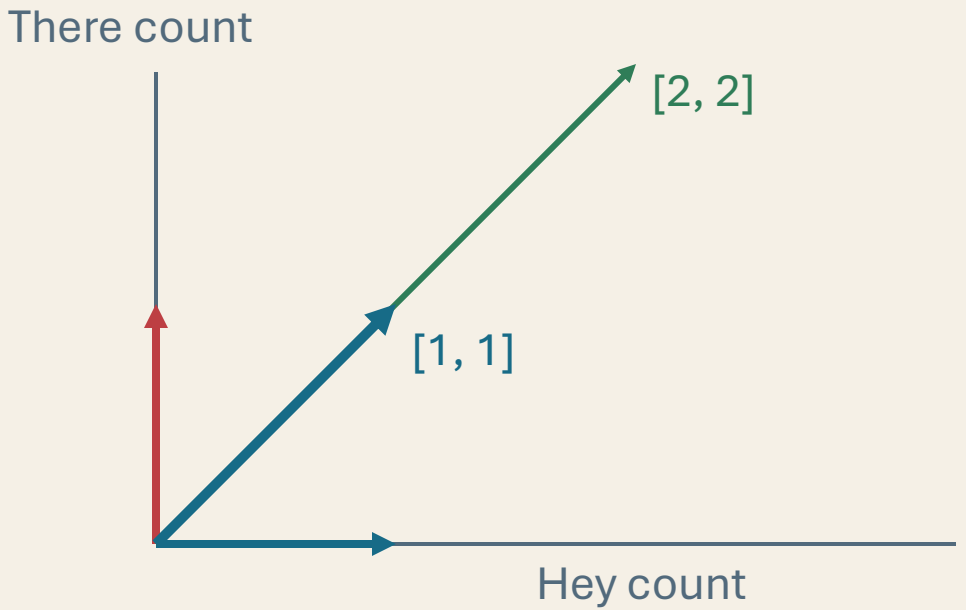
Training adjusts the encoder so useful chunks rank above unrelated chunks.

Karpukhin et al. (2020)

Cosine similarity: direction and length

Cosine similarity compares vector directions. Use word counts first. Vector order: [Hey, There].

Phrase	[Hey, There]
Hey	[1, 0]
There	[0, 1]
Hey there	[1, 1]
Hey hey there there	[2, 2]



Same direction: cosine = 1
Perpendicular directions: cosine = 0

Karpukhin et al. (2020)

Calculating cosine similarity

Cosine similarity measures the alignment of two nonzero vectors, using their angle and ignoring their lengths.

$$\text{cosine}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

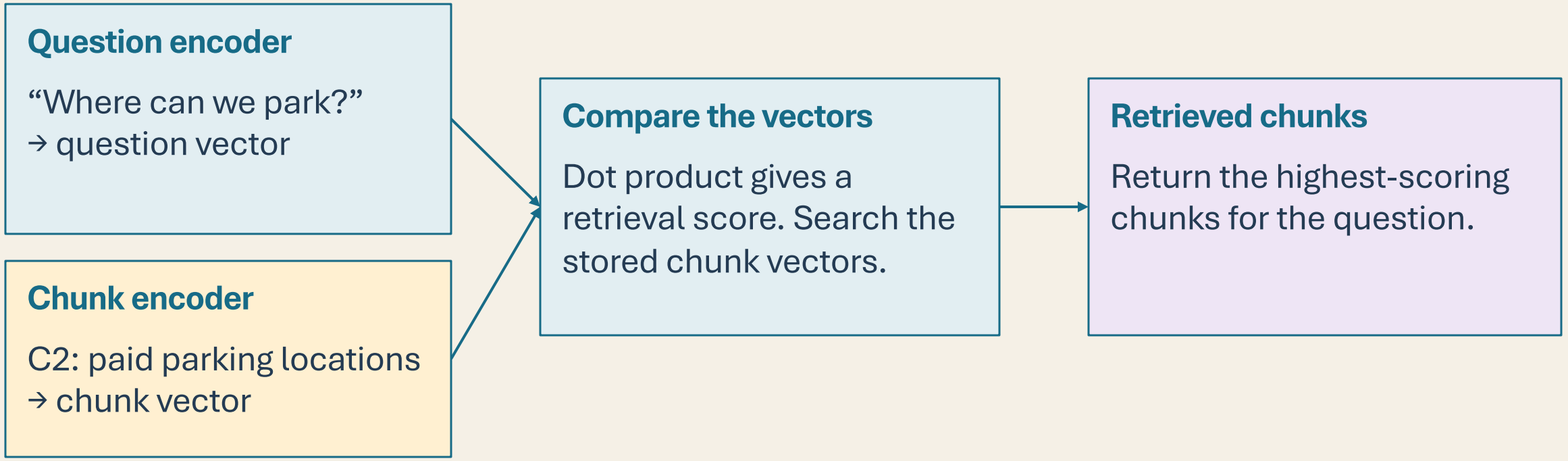
$A \cdot B$: multiply matching coordinates, then add. $\|A\|$ and $\|B\|$: vector lengths.

Vectors A and B	Dot product	Lengths	Cosine
[1, 0] and [1, 1]	$1\times1 + 0\times1 = 1$	1 and $\sqrt{2}$	$1 / \sqrt{2} \approx 0.707$
[1, 1] and [2, 2]	$1\times2 + 1\times2 = 4$	$\sqrt{2}$ and $\sqrt{8}$	$4 / 4 = 1$

These are count vectors for learning the math. RAG can apply the same calculation to learned text embeddings. Opposite directions give −1.
Karpukhin et al. (2020)

DPR: learned retrieval with two encoders

DPR learns separate vector encoders for questions and chunks.



Karpukhin et al. (2020), §§2–3. Native explanation of DPR.

Training DPR: positives, in-batch negatives, and hard negatives

Sample type	Definition and purpose
Positive pair	A question and a chunk labeled relevant. Train this pair to score above the negatives.
In-batch negatives	Chunks paired with other questions in the same batch. Reuse their vectors to supply negatives efficiently.
Hard negatives	High-ranking BM25 chunks that lack the known answer string. These challenge the model with similar wording.

Contrastive learning Raise the positive pair's dot-product score relative to the negatives.

Imperfect labels A negative chunk may still help answer the question.

Evaluation Held-out questions: how often do the top-k chunks contain the answer?

Karpukhin et al. (2020), §§3.2 and 5.2. BM25 ranks chunks using query words.

Build answers from retrieved text

Which retrieved text should enter the model call, and how does it become an answer?

When does query rewriting help?

Optional rewrite: express the request using the facts a source is likely to name.

User request

Can we visit Griffith Observatory at 7 p.m. on Saturday, and where can we park?

Search for the needed facts

Griffith Observatory Saturday building hours; paid parking locations

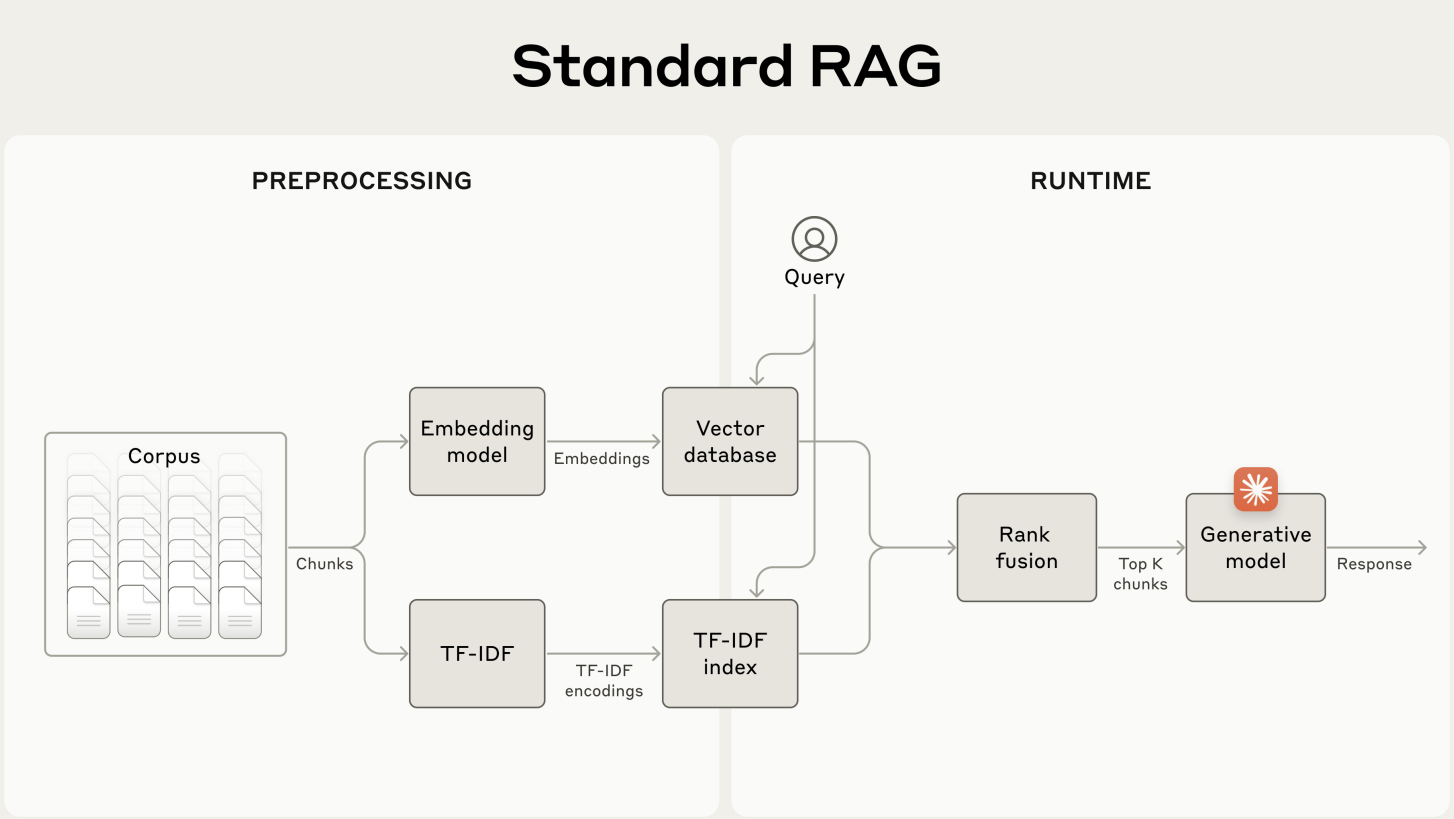
Use the hours to test 7 p.m. Keep Saturday and parking; add no free-parking requirement. Compare retrieval results before claiming the rewrite helps.

Query decomposition: two questions to search

Subquestion	Chunk to find	Search query
Is Saturday 7 p.m. within building hours?	C1 · Saturday hours	Griffith Observatory Saturday building hours
Where can we park?	C2 · Paid parking	Griffith Observatory parking locations

- Decomposition splits a compound question into separate searches. Search for Saturday hours and parking locations separately.
- Combine the results to cover both requested facts. One useful chunk may answer only one part.

Hybrid retrieval: meaning and shared words



Combining routes can recover useful chunks that only one route finds.

[Anthropic (2024), original standard hybrid RAG diagram.]

Two retrieval routes

Embedding search matches meaning.

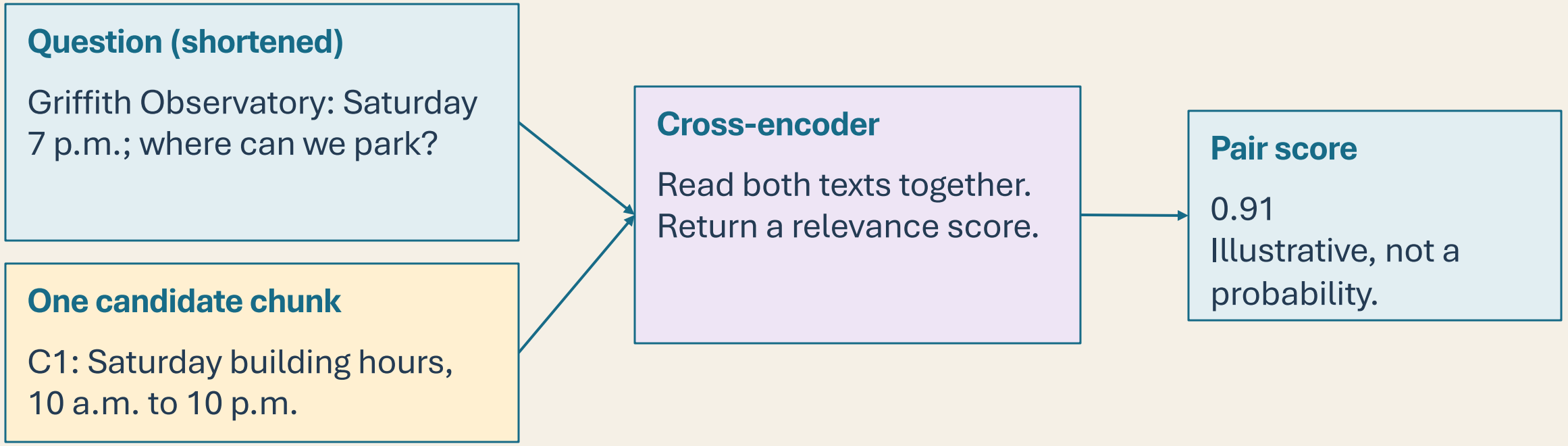
BM25 ranks shared words.

Hybrid search combines their candidate chunks before selecting evidence.

For our question, both routes may find C1 (Saturday hours); keyword search can also find C2 (paid parking). TF-IDF labels the figure's word-weighted keyword index.

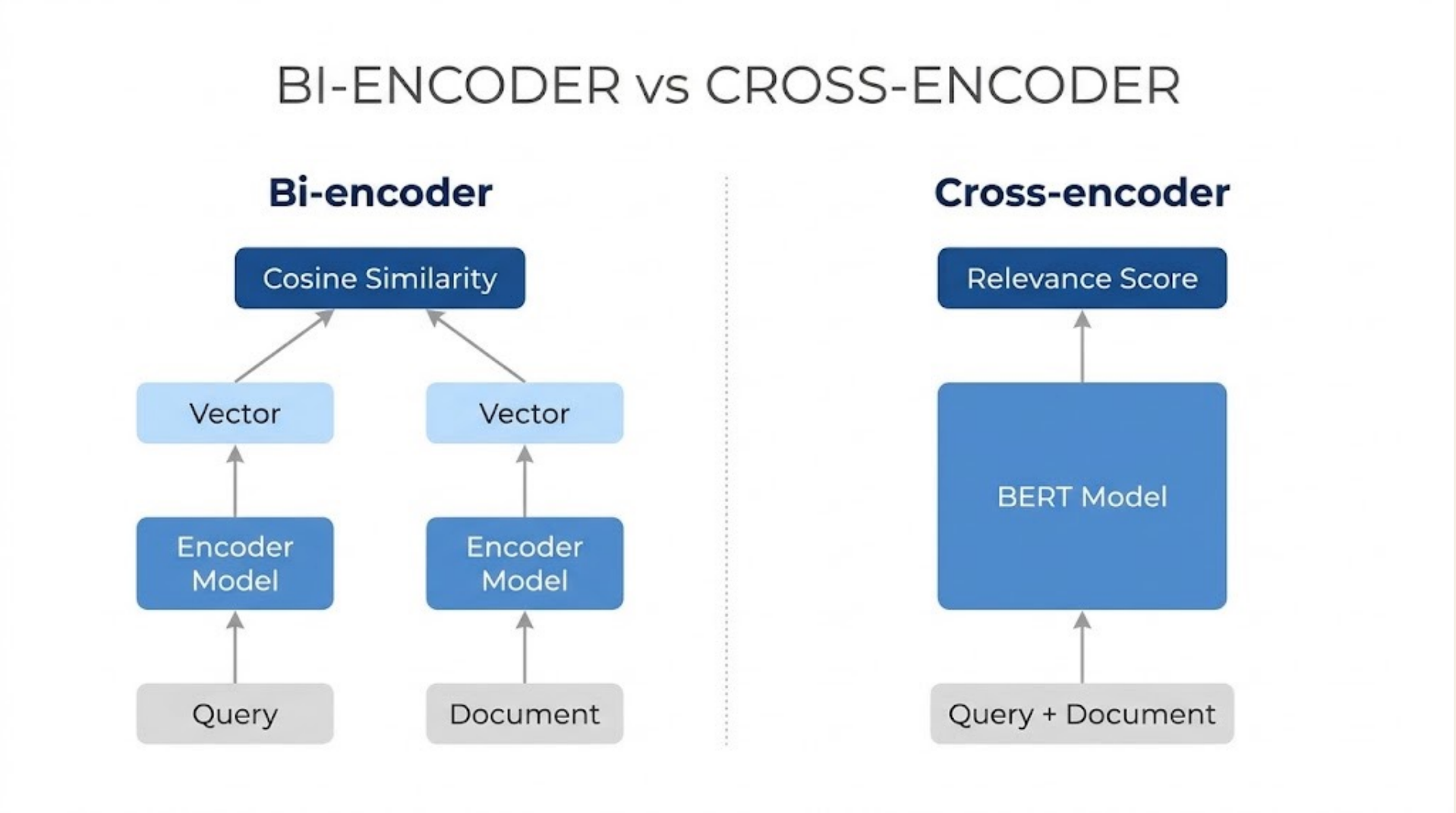
Reranking: What does a cross-encoder read?

A cross-encoder reads the question and one candidate chunk together, then scores their relevance. Repeat for each candidate.



Nogueira & Cho (2019). Joint text scoring differs from separate vector encoding.

What does a cross-encoder read?



From the initial candidate ranking to a new ranking

Initial retrieval supplies candidates. Cross-encoder scores change their order.

Incoming candidate order	After scoring and sorting	
1. C3: Santa Monica Pier location	1. C1: Saturday hours	0.91
2. C1: Saturday building hours	2. C2: parking locations	0.87
3. C2: Observatory parking	3. C3: Pier location	0.14

Illustrative scores. Score a shortlist to limit model calls. Reranking cannot recover a chunk that retrieval missed or prove that an answer is supported.

Reranking orders candidates; the next step selects chunks that fit.

How much room is left for retrieved chunks?

Toy assumption: an 8,000-token limit for input and output together. Actual limits vary by model.

Other input 1,000	Room for retrieved chunks up to 5,000	Answer allowance up to 2,000

8,000 − 1,000 − 2,000 = 5,000 tokens for chunks

Keep evidence for both hours and parking. Include C1 once, rather than wasting space on a duplicate.

5,000 is a ceiling, not a target. Other input includes history, source labels, and formatting.

A token is a unit of model-processed text. All numbers here are illustrative.

Check answers and citations

Does each claim follow from the source it cites?

A model can write a plausible answer even when a citation is irrelevant or a claim is unsupported.

Separate retrieval quality, answer quality, citation support, and source freshness.

Relevance and support

Claim to check: “The listed parking options are free.”

C1: Building hours “Saturday building hours: 10 a.m. to 10 p.m.”

C2: Paid parking “Paid parking is available in the Observatory lot and along West Observatory Road.”

Source chunk	Relevant to parking cost?	Supports the claim?
C1: Building hours	No	No information about parking cost.
C2: Paid parking	Yes	No. It explicitly describes paid parking.

A chunk can be relevant to a claim without supporting it.

Teaching excerpts: LA Visitor Notes, saved 21 Oct 2026. Gao et al. (2023).

One sentence, two claims to check

Source chunk C1: Griffith Observatory hours

“Saturday building hours: 10 a.m. to 10 p.m.”

Invented draft: “The building is open at 7 p.m. on Saturday, and a **parking space is guaranteed** [C1].”

Hours claim: supported

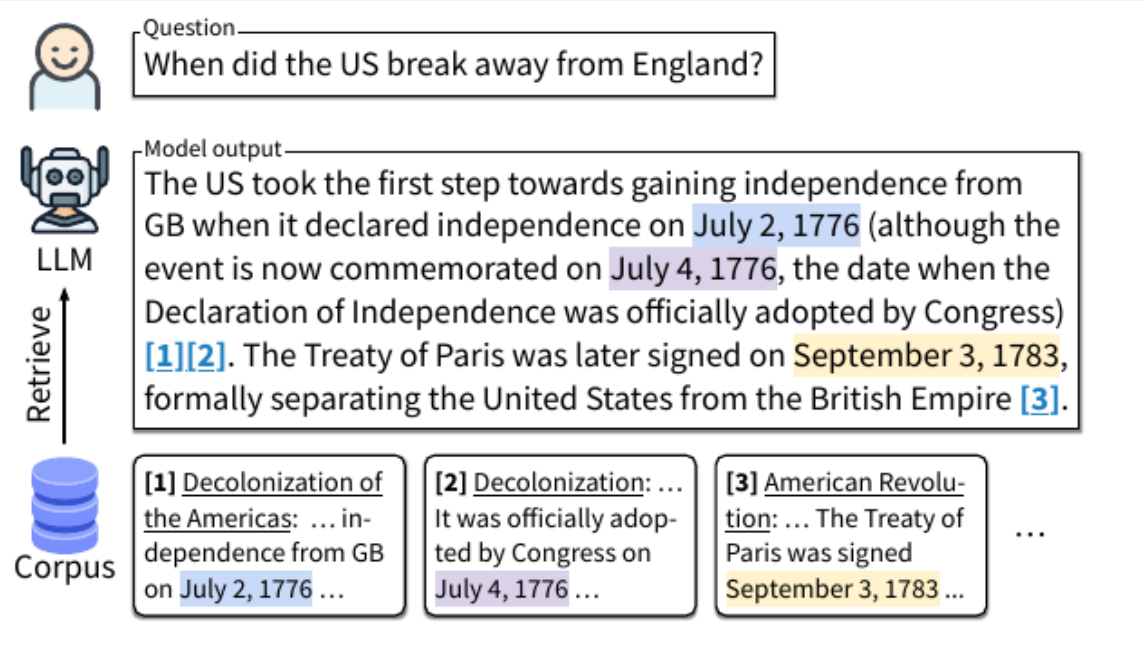
7 p.m. falls within C1’s Saturday hours.

Parking guarantee: unsupported

C1 says nothing about available spaces. It does not prove the lot is full either.

ALCE: evaluating answers with citations

ALCE benchmarks answer quality and the evidence supplied by citations.



Task: answer a factual question using a document collection and identify the sources for the claims.

Note that a citation marker alone does not prove support.

Gao et al. (2023), complete Figure 1. ALCE = Automatic LLMs' Citation Evaluation.

Three benchmarks test three kinds of answers

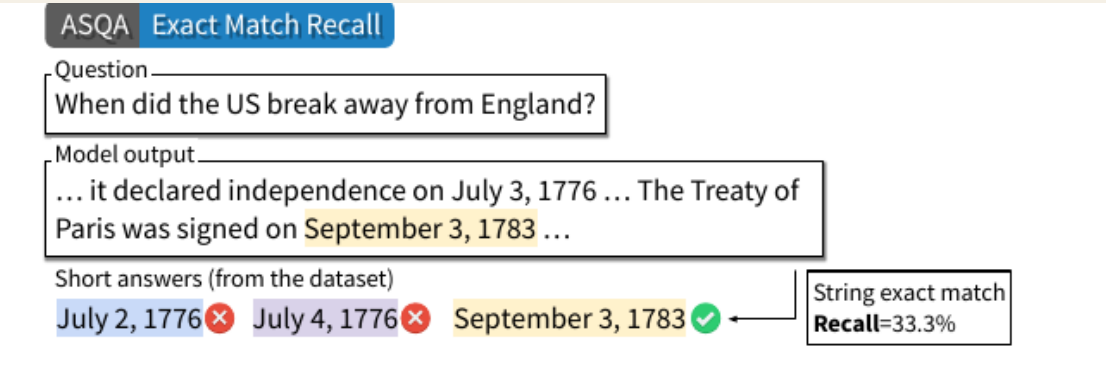
A benchmark uses test questions and reference answers to compare systems.
ALCE adapts three question-answering datasets to evaluate answers with citations.

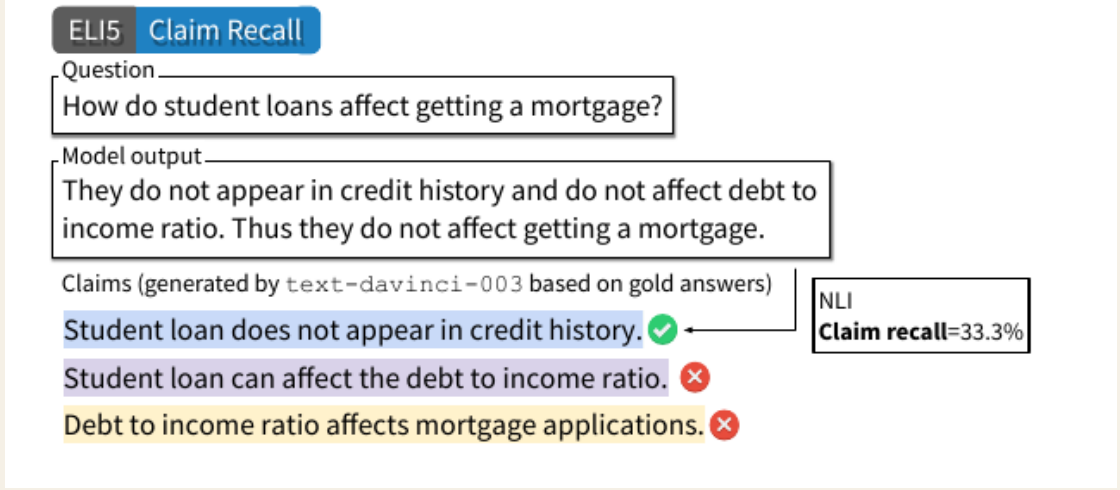
ASQA Ambiguous questions	A dataset of factual questions with several interpretations. A complete answer covers them. Check: Does it include the expected short answers?
QAMPARI Lists of answers	A dataset of questions answered by lists of items, such as people or films, drawn from several sources. Check: Which listed items are correct, and which expected items are missing?
ELI5 Detailed explanations	A dataset of why/how/what questions requiring detailed explanations. ELI5 means “Explain Like I’m Five.” Check: Does it cover the key facts in the reference answer?

Correctness depends on the task. Citation checks ask whether the sources support the claims.

Gao et al. (2023), §§2–3. ALCE checks correctness, citation quality, and fluency.

Correctness: reference answers and claims





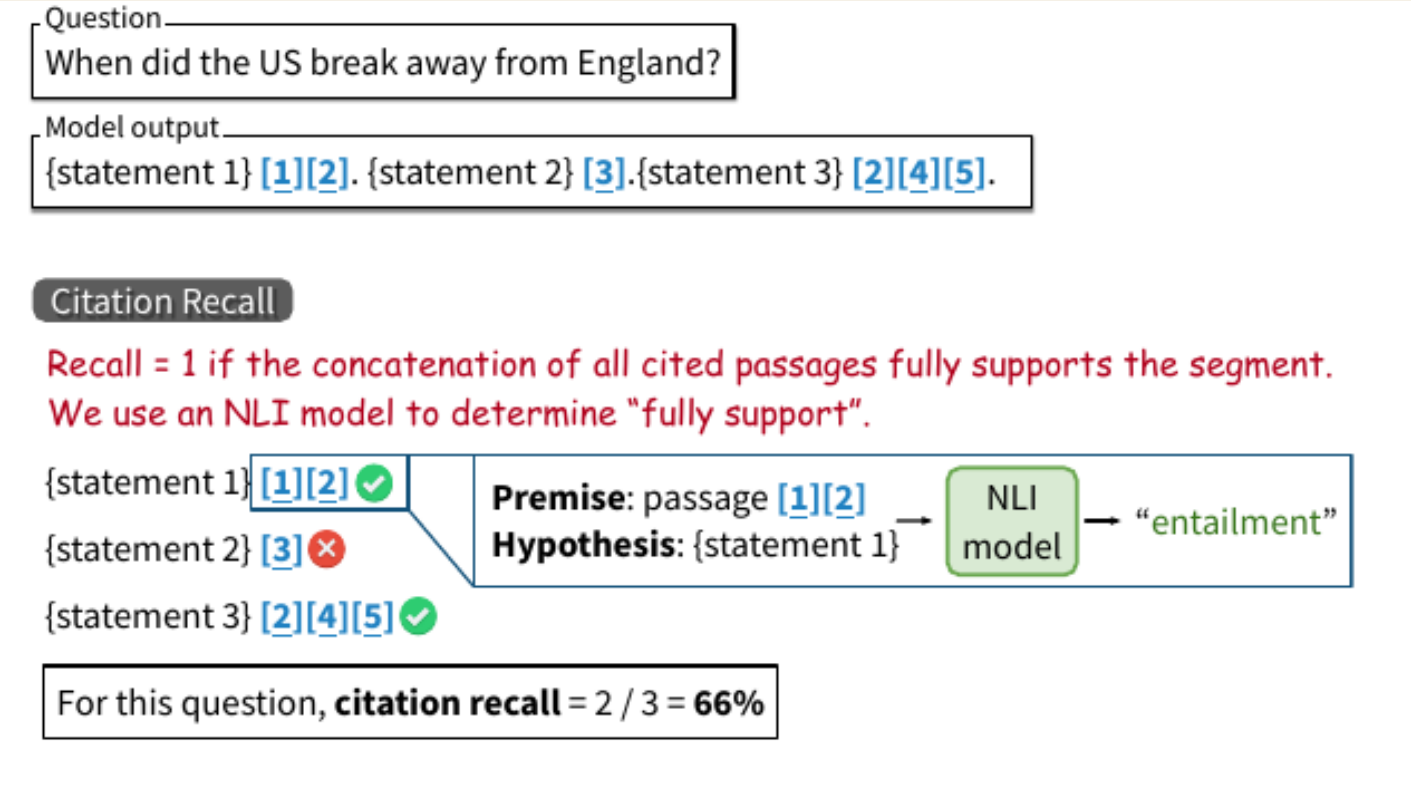
ASQA: the answer includes one of three reference dates. Exact-match recall = 1/3.

ELI5: an NLI model checks whether the answer supports each reference claim. Here, 1/3 are covered.

NLI here means a model judges whether one text supports a statement.

Gao et al. (2023), Figure 2 excerpts. Correctness coverage differs from citation support.

Citation recall: supported statements



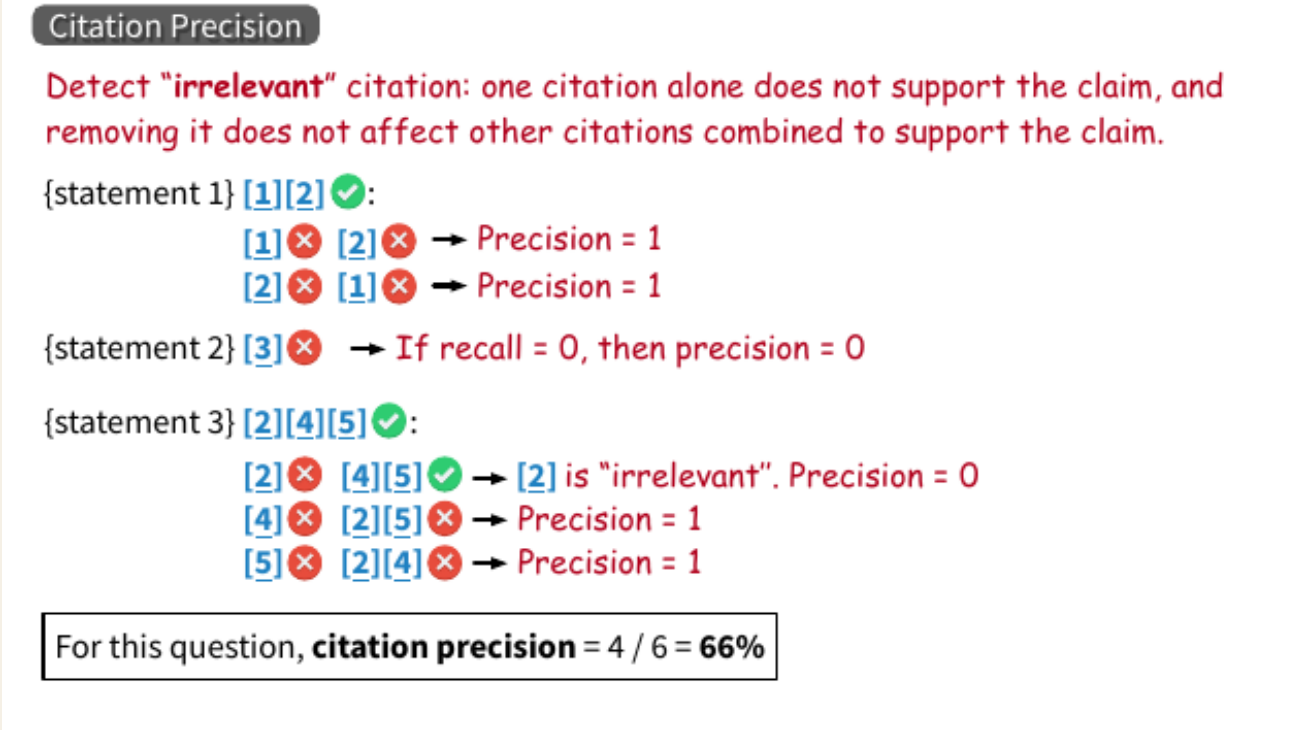
Do the citations support each statement?

Citation recall:
the fraction of answer statements fully supported by their cited source chunks together.

Two of three statements pass: 2/3 = 66.7% (the figure displays 66%).
An uncited statement receives no citation support.

Gao et al. (2023), Figure 3 recall panel. NLI estimates source-to-claim support.

Citation precision: useful citations



The Top Line ({statement 1} [1][2] ✅): This means that citations [1] and [2] *working together* successfully prove the statement.
Precision =0 or 1 in each line is a binary metric meaning irrelevant or relevant

Which citations are irrelevant?

Citation precision:
checks individual citation occurrences.

A source is irrelevant if it neither supports the claim alone nor contributes to the sources' joint support.

Four of six citations pass: 4/6 = 66.7% (the figure displays 66%).

Gao et al. (2023), Figure 3 precision panel. Unsupported statements receive zero precision.

Correct content and citation support can diverge

AI can generate factually correct answers, but fail to provide accurate citations to support those answers

	Fluency	Correct.	Citation	
	(MAUVE)	(Claim)	Rec.	Prec.
ChatGPT				
VANILLA (5-psg)	57.2	12.0	51.1	50.0
w/ RERANK	56.1	11.4	**69.3**	**67.8**
SUMM (10-psg)	40.3	12.5	51.5	48.2
SNIPPET (10-psg)	62.9	14.3	50.4	45.0
w/ INTERACT	68.0	13.3	47.8	45.0
INLINESEARCH	49.7	13.4	45.6	43.7
CLOSEDBOOK	32.6	**18.6**	15.5	15.5

Correct content and support diverge

ELI5 explanation questions, 2023 ChatGPT.

VANILLA: reference-claim recall 12.0; citation recall 51.1.

CLOSEDBOOK writes without retrieved sources, then adds citations afterward: claim recall 18.6, citation recall 15.5.

Just because an LLM successfully writes correct information does not guarantee that the citations it outputs actually support what it wrote.

Gao et al. (2023), Table 6, ChatGPT rows. MAUVE is a text-distribution proxy for fluency.

When the sources cannot answer a follow-up

Follow-up question

“Will a parking space be available when we arrive at 7 p.m.?”

What C2 actually says

Paid parking is in the Observatory lot and along West Observatory Road.
This describes locations, not live occupancy.

A supported response

“These sources list parking locations, but cannot confirm whether a space will be available at 7 p.m.”

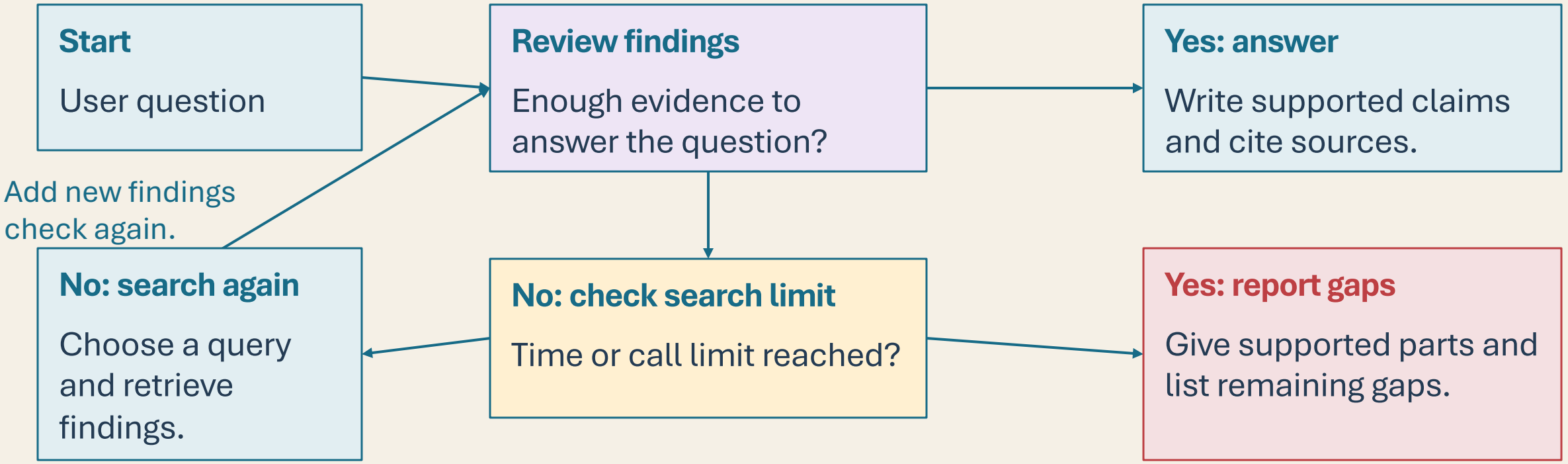
Abstention means leaving an unsupported claim unanswered. A further search may still find nothing.

Agentic RAG: choosing the next search

When should the agent search again, and what should it search for?

Search again, answer, or report the gaps

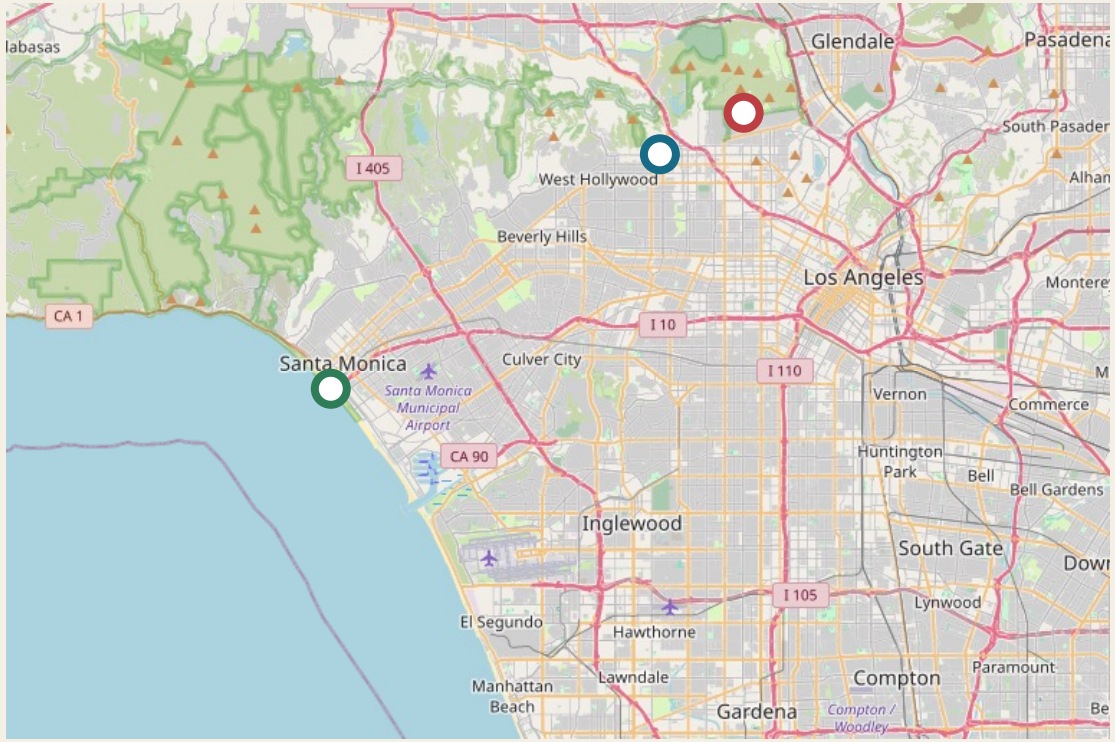
The agent checks its evidence before deciding whether another search is needed.



Example limits: three search calls or 30 seconds. Enough evidence can end the loop earlier.

Where are our three LA stops?

New request: Can we visit all three stops in one day without a car?



Griffith Observatory

Griffith Park, north of central LA

Hollywood Boulevard

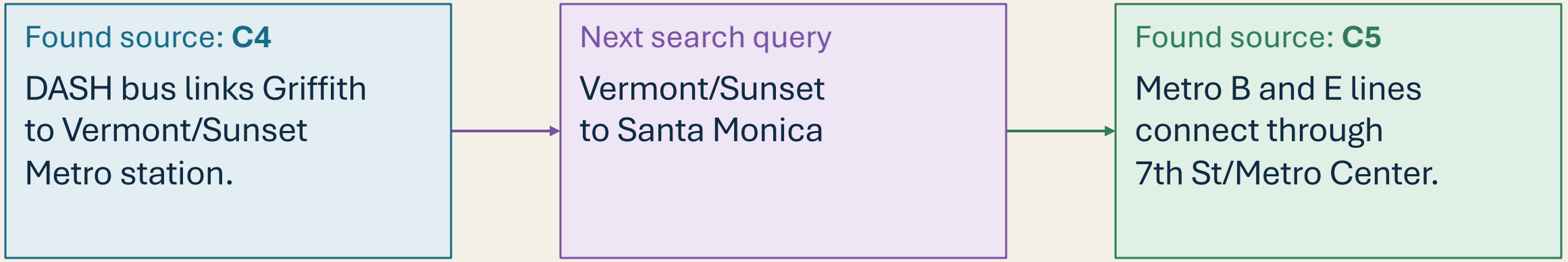
Hollywood, south of Griffith Park

Santa Monica Pier

The Pacific coast, west of the other stops

Use the first result to choose the next search

Task: Plan the three-stop LA visit without a car. The first result gives a clue for the next search.

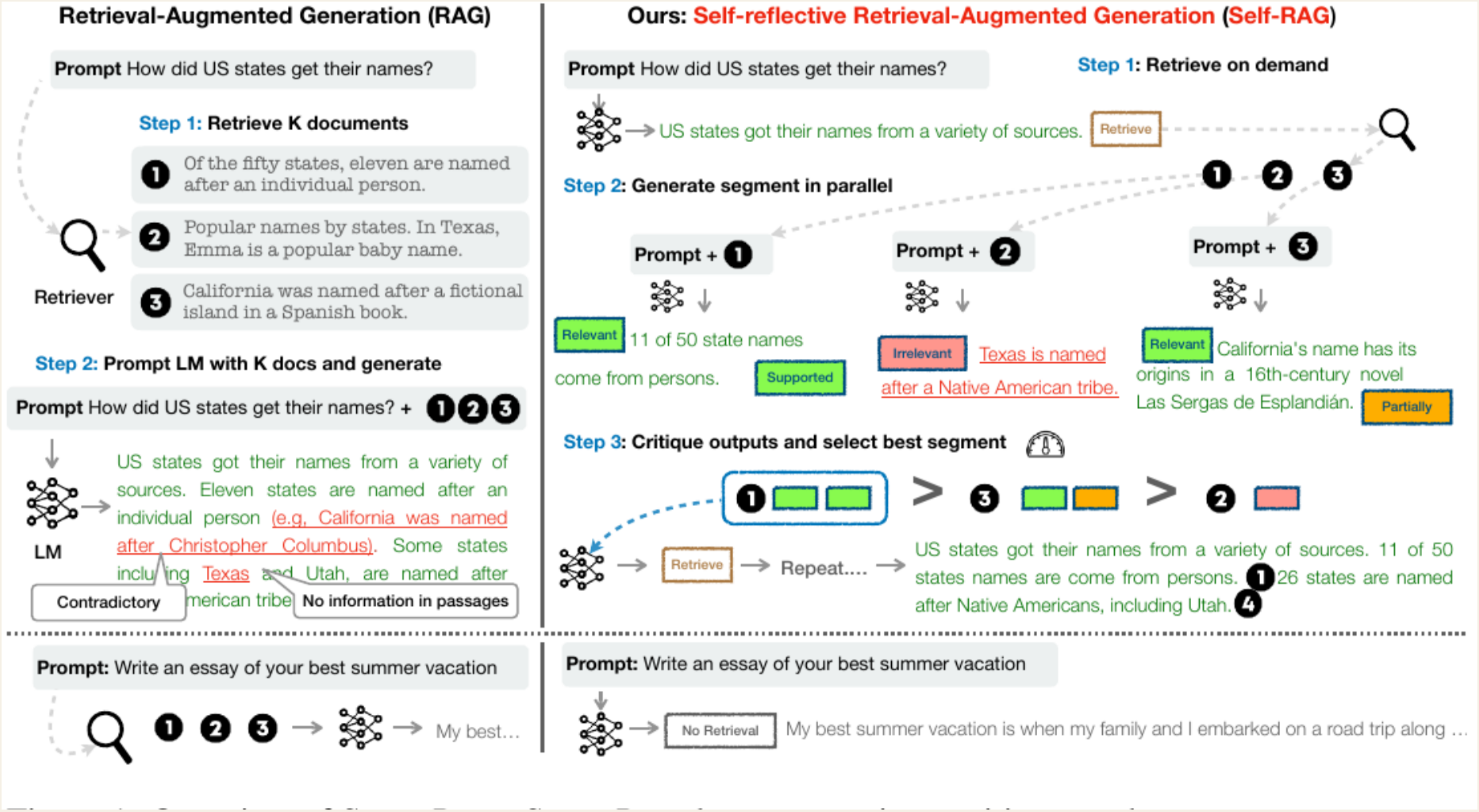


Keep C4's station name in the working context. Combine it with the destination for the next query.

Still missing: travel times, the Hollywood leg, and time at each stop. A connection alone does not prove the day is feasible.

Adapted sources: Griffith transit ; Metro B/D map Metro E map . Saved 21 Oct 2026.

Self-RAG: learned retrieval and self-assessment



Asai et al. (2024), complete Figure 1. Self-RAG learns retrieval and self-assessment.

Self-RAG: factuality and citation results

Selected models and two metrics from the paper's Table 2. Higher is better.

Model	PopQA Answer accuracy	ASQA Citation precision
Ret-ChatGPT	50.8	65.1
Our SELF-RAG 7B	54.9	66.9
Our SELF-RAG 13B	**55.8**	**70.3**

Ret-ChatGPT: ChatGPT with retrieved sources.
7B and 13B: model size in billions of parameters.

Why this design may help

Retrieve when needed.

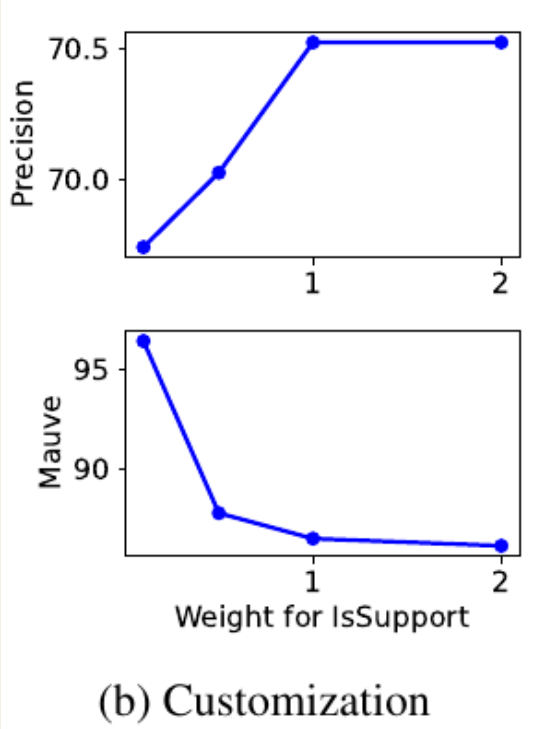
Score candidate answer segments for relevance, source support, and usefulness.

Use these learned checks to choose what to write next.

Note: Because these models differ in size and training, we cannot prove these gains are solely due to the Self-RAG architecture.

Asai et al. (2024), Table 2 excerpt. Citation precision checks for irrelevant citations.

The support–fluency tradeoff in Self-RAG



(b) Customization

Favoring support changes the output

ASQA, Self-RAG 7B. Raising the weight for the support label increases citation precision.
MAUVE falls, indicating a change in generated-text distribution relative to references.
MAUVE is not a factuality score.

Learned reflection offers a control knob, with a measured tradeoff rather than a free improvement.

Asai et al. (2024), Figure 3(b). Precision and MAUVE use different axes.

Alternatives and design choices

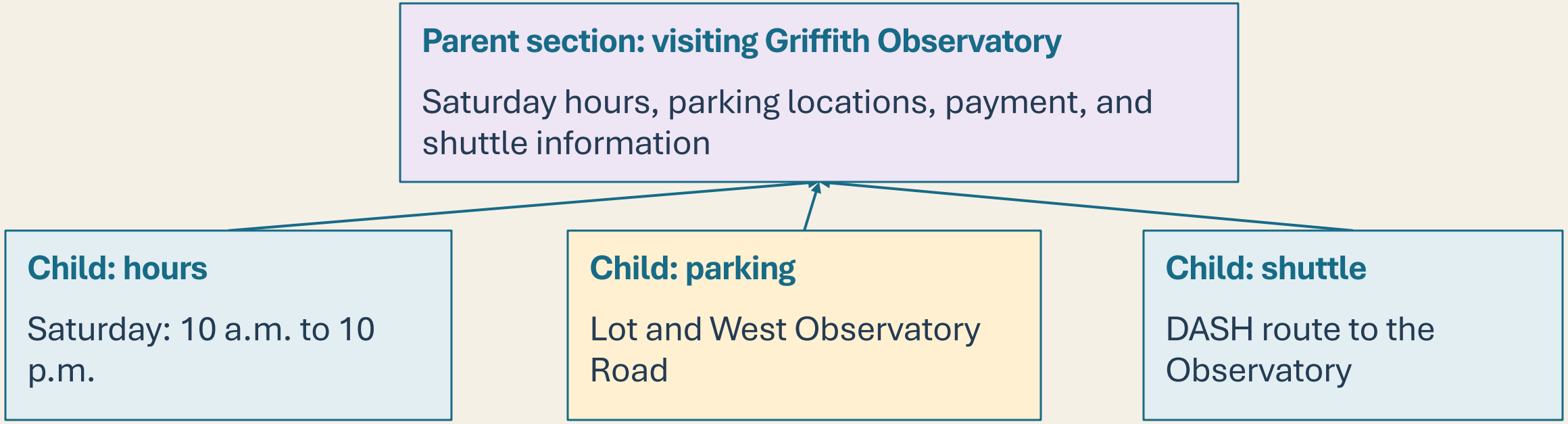
Which approach fits the evidence the question requires?

We have seen one RAG pass and searches that adapt to earlier findings.

Compare ways to preserve context, follow relationships, use full documents, and inspect large sources.

From a matching chunk to its parent section

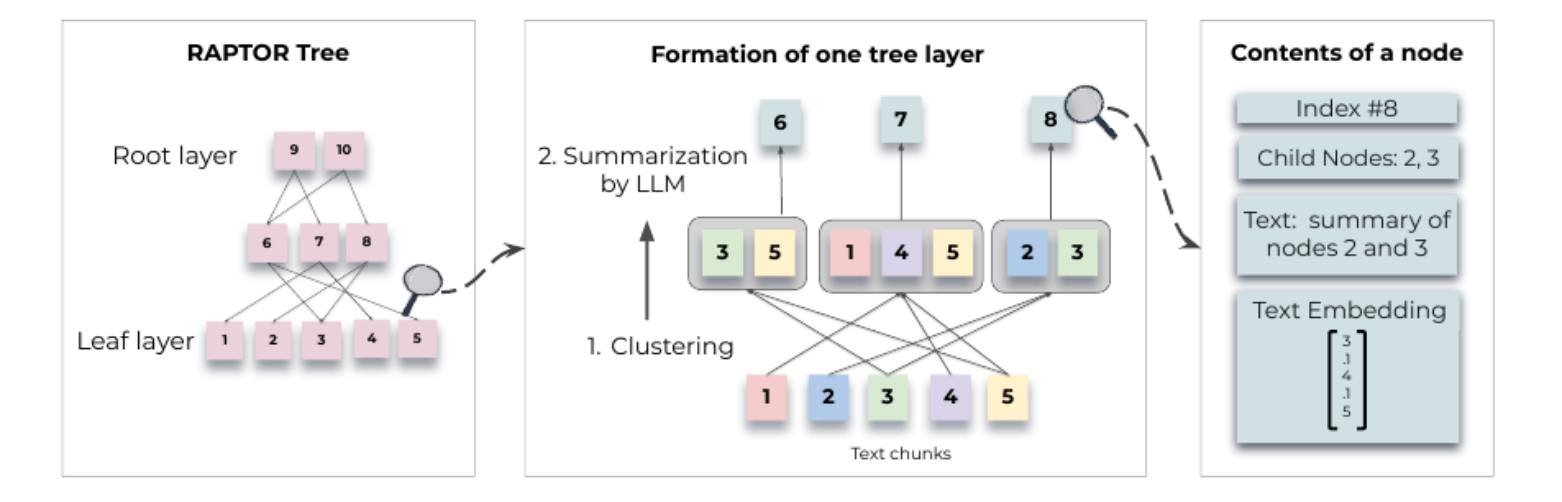
Hierarchical retrieval links a small chunk to a larger parent section. Retrieve the small match, then read enough of its parent to interpret it.



This follows document structure. RAPTOR next learns a different hierarchy by clustering text.

RAPTOR: searchable summaries at several levels

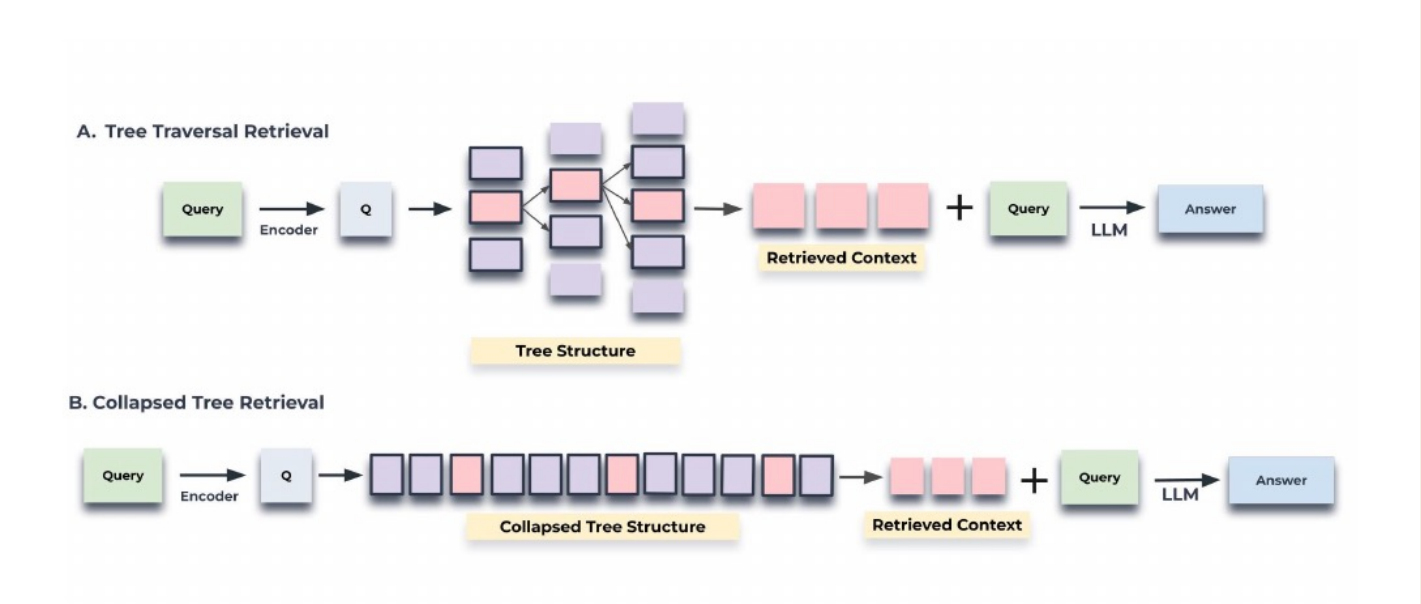
RAPTOR builds searchable layers of summaries to combine local details with broader context.



Cluster → summarize → repeat. Node 8 summarizes chunks 2 and 3.
Summary nodes keep child links and an embedding. Clusters may overlap.

Sarthi et al. (2024), complete Figure 1.

RAPTOR's two retrieval strategies



Two ways to search the tree

A: Tree traversal starts with broad summaries, then searches selected children.
B: Collapsed-tree retrieval searches chunk and summary nodes together across all levels.
The main experiments use B. A root-first search is not required.

A query can retrieve a broad summary, a detailed chunk, or both.

Sarthi et al. (2024), complete Figure 2. Boxes are searchable chunk or summary nodes.

RAPTOR's controlled QASPER comparison

Retriever	GPT-3 F-1 Match	GPT-4 F-1 Match	UnifiedQA F-1 Match
Title + Abstract	25.2	22.2	17.5
BM25	46.6	50.2	26.4
DPR	51.3	53.0	32.1
RAPTOR	**53.1**	**55.7**	**36.6**

Same reader, different retrieval

QASPER asks questions about research papers. Answer F1 measures overlap with reference answers.
With GPT-4 as the reader:
BM25: 50.2
DPR: 53.0
RAPTOR: 55.7
RAPTOR gains 2.7 F1 over DPR here.

Searchable summaries helped this dataset; answer F1 is not the fraction of fully correct answers.

Sarthi et al. (2024), complete Table 3. Reader model held fixed within each column.

Restore the context a chunk lost

A chunk can lose the heading that tells us what its rule applies to.

Without document context

“Pay stations take credit cards only.”

Which place? Which service?

With chunk-specific context

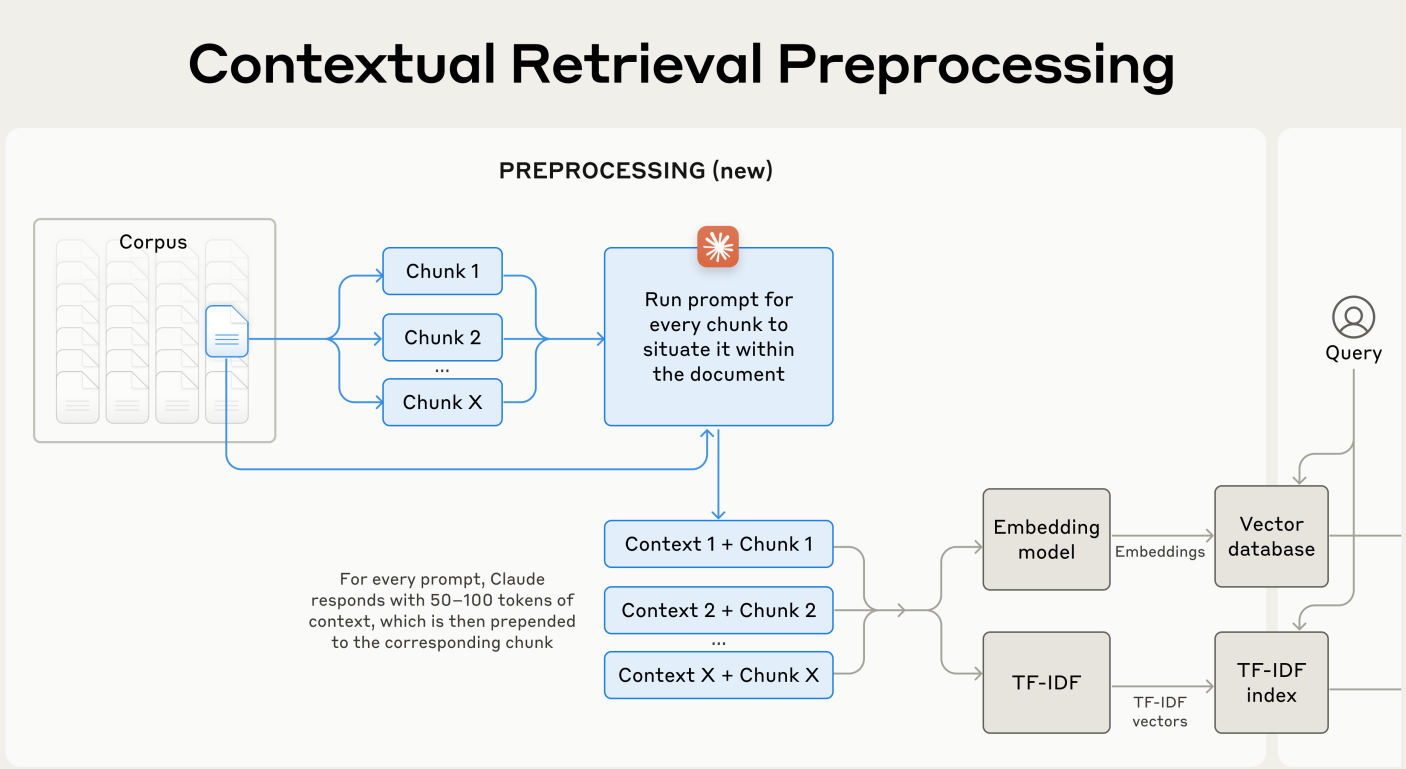
“Griffith Observatory: paid parking.”

“Pay stations take credit cards only.”

Add a short explanation from the document before indexing the chunk. Verify the added text: a wrong explanation can make retrieval worse.

LA teaching example for Anthropic’s Contextual Retrieval (2024). Original chunk stays intact.

Contextual Retrieval: explain before indexing



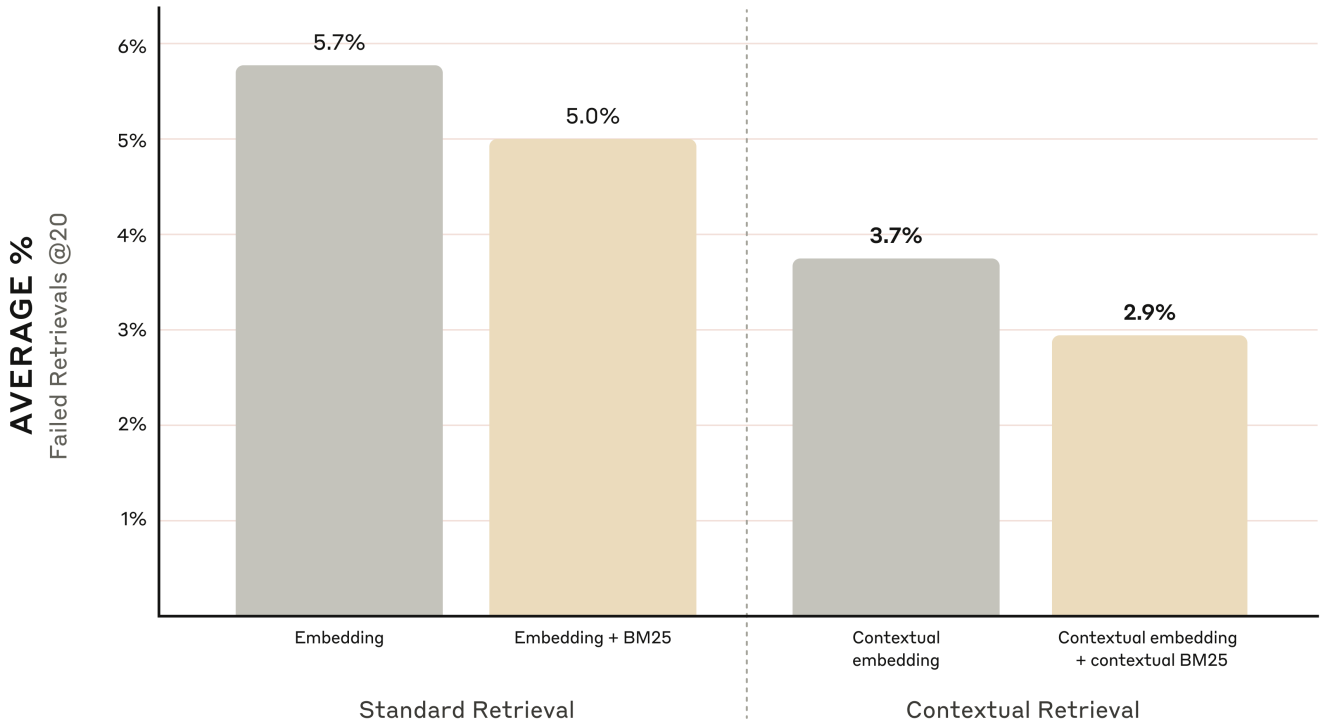
Contextual Retrieval: add context first

An LLM reads the document and writes a short explanation of each chunk's role. Prepend it to that chunk.
Contextual embeddings encode this enriched text.
Contextual BM25 applies keyword ranking to the same enriched text.

Add a short document explanation before embedding and keyword indexing.

[Anthropic (2024), complete original preprocessing diagram.]

Contextual text reduced missed evidence



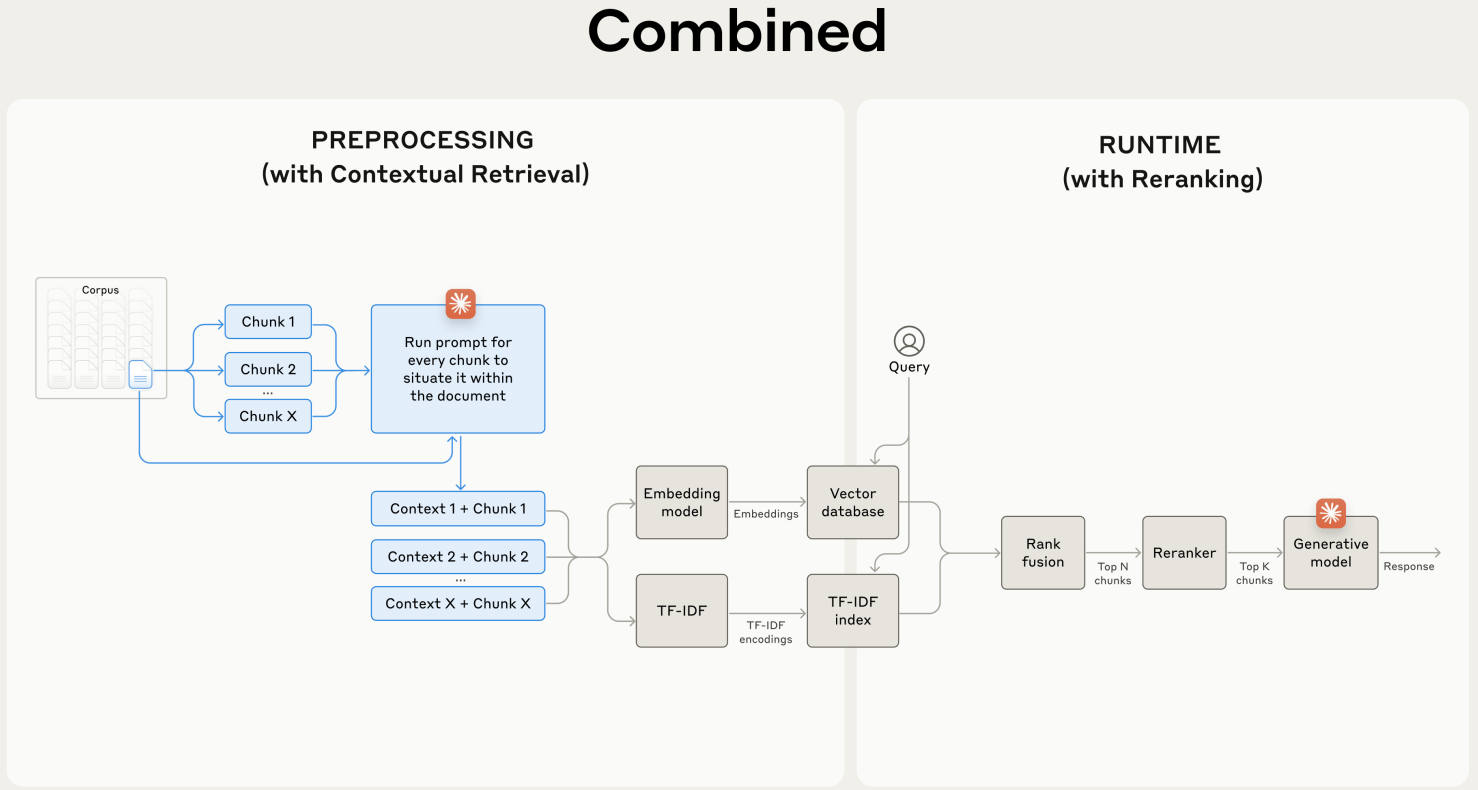
Fewer relevant chunks were missed

Metric: 1 – recall@20, the share of relevant evidence missed in the top 20 chunks. Lower is better.
Standard embeddings: 5.7%
Standard hybrid: 5.0%
Contextual embeddings: 3.7%
Contextual hybrid: 2.9%

The 35% and 49% reductions are relative to 5.7%, not percentage-point or sequential gains.

Anthropic (2024), complete original four-bar result plot. This is not final-answer accuracy.

Contextual retrieval followed by reranking



Retrieve widely, then rerank

Contextual hybrid retrieval finds candidates. A reranking model reads each candidate with the question and scores relevance again.

Reported setup: 150 candidates → 20 selected chunks. These counts are experimental choices.

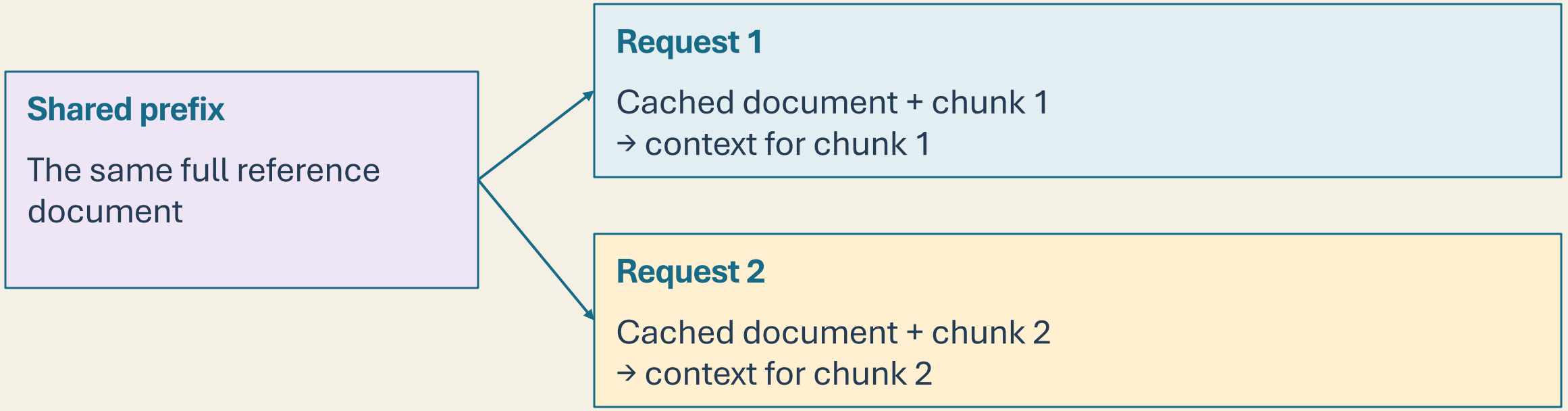
Missed evidence: 5.7% → 1.9%, a 67% relative reduction.

The 1.9% result is a separate reranking experiment, absent from the previous four-bar plot.

Anthropic (2024), complete original combined pipeline and reported reranking result.

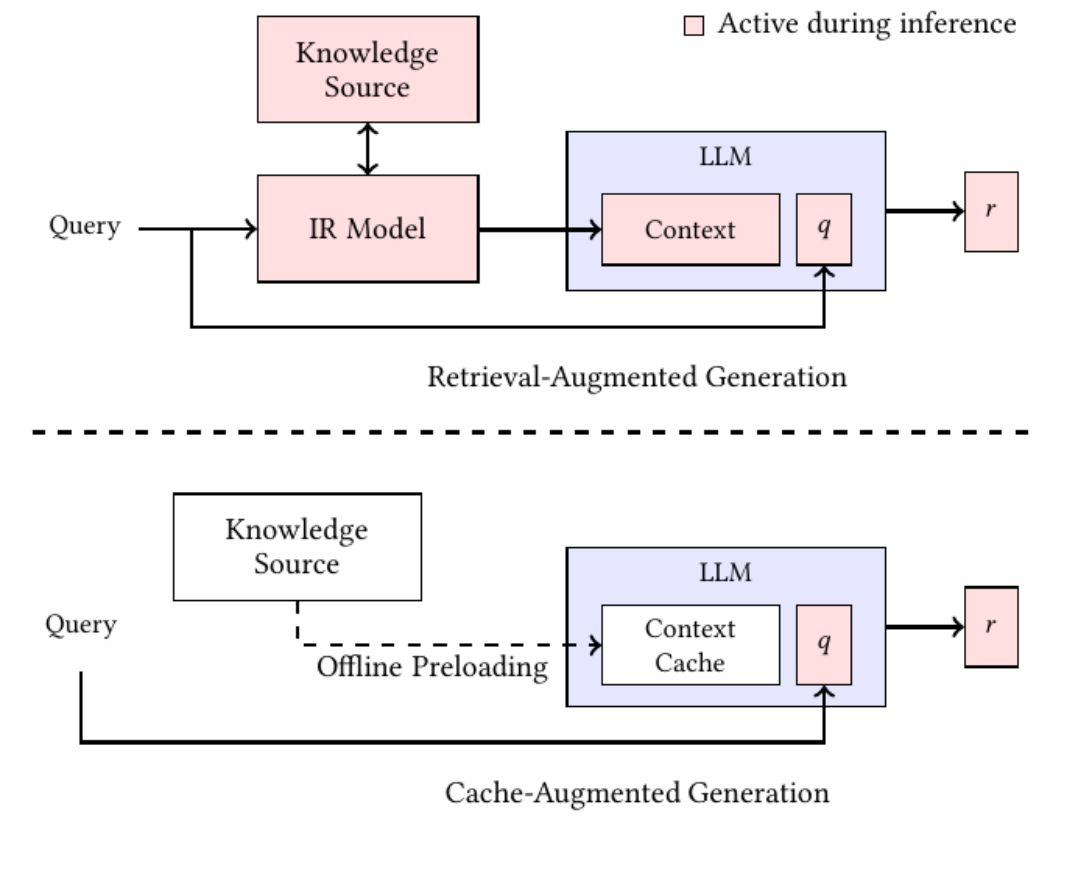
Reuse document processing during preprocessing

Prompt caching reuses computation for a repeated input prefix



Anthropic (2024), “Using Prompt Caching to Reduce the Cost of Contextual Retrieval”.

CAG: reuse the collection across questions



Cache-Augmented Generation (CAG)

Load the whole collection into context and reuse its processed prefix for later questions.

Top: IR (information retrieval) selects a subset.
Bottom: CAG preloads the full collection. q = query; r = response.
Use this when the collection fits and stays useful. Each call still has a finite limit.

Whole-context answering removes query-time retrieval, while keeping context and update costs.

Chan et al. (2025), complete Figure 1. CAG is one whole-context approach.

CAG results on a small collection

Size	System	Top-k	HotPotQA BERT-Score	SQuAD BERT-Score
Small	Sparse RAG	1	0.6788	0.7214
		3	0.7626	0.7616
		5	0.7676	0.7608
		10	0.7521	0.7584
	Dense RAG	1	0.7164	0.6216
		3	0.7582	0.7106
		5	0.7481	0.7334
		10	0.7576	0.7586
	CAG (Ours)		**0.7951**	**0.7695**

A small collection can favor full context

Llama 3.1 8B;
HotPotQA small setting: 16 documents, about 21,000 tokens.
CAG: BERTScore 0.7951.
Dense RAG, top 3: 0.7582.

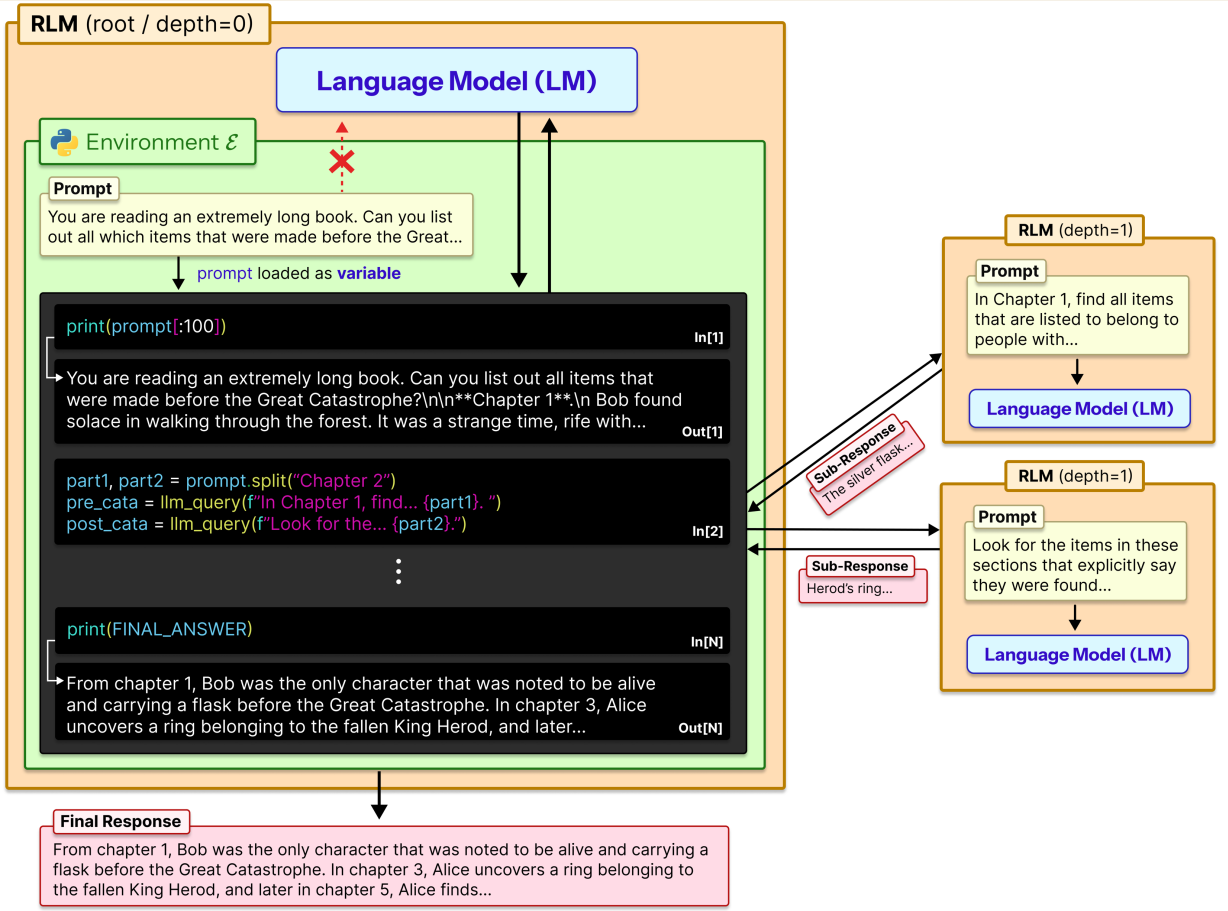
BERTScore measures similarity to a reference answer; it is not factual correctness as a percentage.

Dense: semantic search. Sparse: keyword search

Chan et al. (2025), original Table 2, small-collection rows. Larger settings differ.

RLM: make a long input programmable

RLM keeps long inputs in a programming environment, then uses code and model sub-calls to analyze them.



RLM: Recursive Language Model.

REPL: a programming workspace that retains variables.

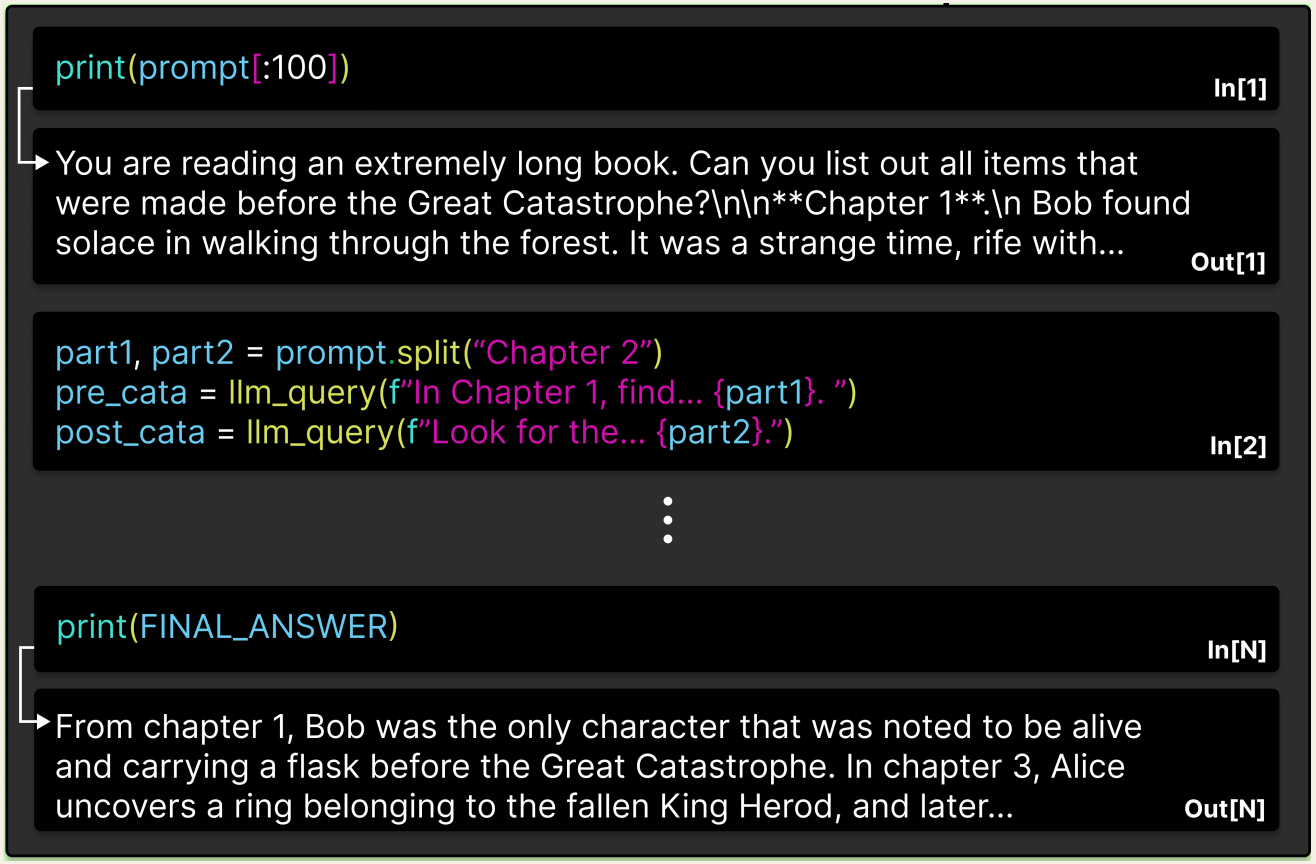
Root model: the main controller.

Sub-call: a model invocation on selected text.

The long input is a variable, not all in the root model's context.

Zhang, Kraska & Khattab (2026), complete Figure 2. Pinned arXiv v3.

From stored input to model sub-calls



What the code does

1. Inspect the stored input.

2. Split it into sections.

3. Send selected sections to model sub-calls.

4. Combine the returned findings into the final answer.

Experiment depth 0: code, no sub-calls. Depth 1: model sub-calls. Greater depths allow nested RLM calls. Figure labels count the root as level 0.

Zhang et al. (2026), Figure 2 detail. GPT-5 root experiments use GPT-5-mini sub-calls.

RLM results depend on the task

Model	CodeQA	BrowseComp+ (1K)	OOLONG	OOLONG-Pairs
Task Length N (tokens)	23K-4.2M	6M-11M	131K	32K
GPT-5 (with RLM sub-calls to GPT-5-mini)				
Base Model	24.0* ($0.13 ± $0.07)	0.0* (N/A) ± (N/A)	44.0 ($0.14 ± $0.02)	0.1 ($0.16 ± $0.10)
CodeAct (+ BM25)	22.0* ($0.06 ± $0.08)	51.0 ($0.71 ± $1.20)	38.0 ($0.61 ± $1.06)	24.7 ($0.75 ± $0.43)
CodeAct (+ sub-calls)	24.0* ($0.06 ± $0.08)	0.0* (N/A) ± (N/A)	40.0 ($0.85 ± $1.27)	28.4 ($1.11 ± $0.62)
Compaction agent	58.0 ($1.31 ± $1.46)	70.5 ($0.57 ± $0.10)	46.0 ($0.13 ± $0.01)	0.1 ($0.13 ± $0.09)
OpenCode	18.0* (N/A) ± (N/A)	0.0* (N/A) ± (N/A)	32.0 (N/A) ± (N/A)	3.1 (N/A) ± (N/A)
OpenCode (+ context offloading)	64.0 (N/A) ± (N/A)	**94.0** (N/A) ± (N/A)	52.0 (N/A) ± (N/A)	4.8 (N/A) ± (N/A)
RLM (recursion depth=0)	58.0 ($0.18 ± $0.56)	88.0 ($0.44 ± $0.90)	36.0 ($0.37 ± $0.42)	43.9 ($0.69 ± $1.16)
RLM (recursion depth=1)	62.0 ($0.11 ± $0.10)	91.3 ($0.99 ± $1.22)	56.0 ($0.43 ± $0.85)	58.0 ($0.33 ± $0.20)
RLM (recursion depth=2)	**66.0** ($0.15 ± $0.30)	92.0 ($0.55 ± $0.69)	56.5 ($1.10 ± $3.25)	65.5 ($0.33 ± $0.44)
RLM (recursion depth=3)	58.0 ($0.15 ± $0.27)	92.0 ($0.51 ± $0.54)	**58.0** ($0.51 ± $0.54)	**76.0** ($0.39 ± $0.32)

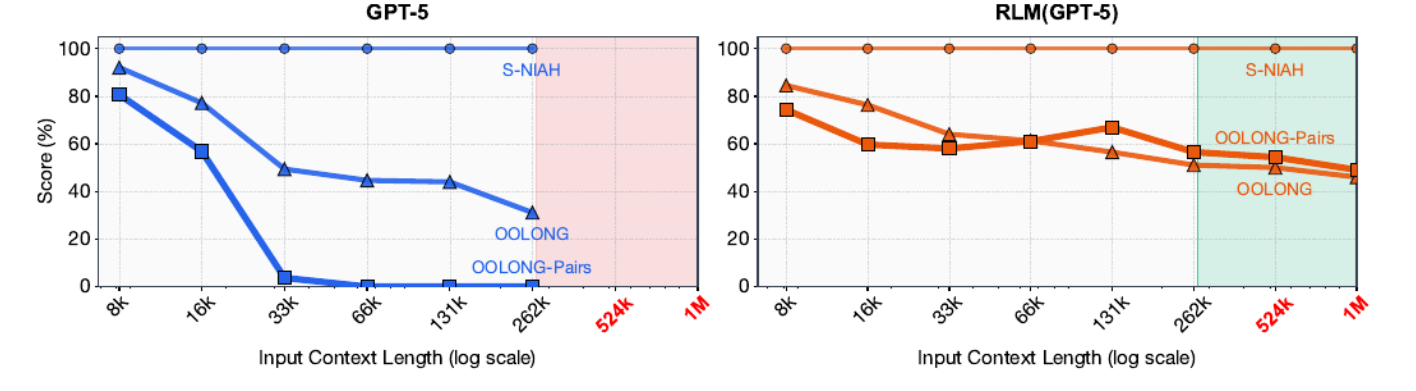
Compare the GPT-5 systems

OOLONG-Pairs aggregates pairs from many records. Score: list F1.
CodeAct + BM25: 24.7
RLM without sub-calls: 43.9
RLM depth 1: 58.0
BrowseComp+ offline search accuracy: depth 1 RLM 91.3%; OpenCode with file offloading 94.0%.

RLM helped dense aggregation here. Another agent scored higher on the offline search task.

Zhang et al. (2026), original Table 1 GPT-5 block. CodeAct = coding agent with BM25 search.

RLM across input lengths



Length is only part of the difficulty

The plots compare GPT-5 and RLM as the supplied input grows.
S-NIAH: find a single item.
OOLONG: aggregate records.
OOLONG-Pairs: reason over pairs.
The red area starts beyond the tested GPT-5 window, 272K tokens.

An external input can exceed one call's window. Processing every needed relation remains difficult.

Zhang et al. (2026), complete Figure 1. Each model call still has a finite context.

RAG, agentic retrieval, and RLM

These approaches overlap. RLM is an agentic pattern for processing a supplied long input.

Approach	What chooses the next information?	Where the information lives
Single-pass RAG	A retrieval query selects chunks once	External index; selected chunks enter the prompt
Agentic RAG	The agent reviews findings and chooses further queries	Sources and working context across searches
RLM	Generated code inspects stored input and makes sub-calls	External variables and results; selected text enters calls

Ordinary agents can also use files, code, and sub-calls. RLM's specific combination makes the long supplied input and intermediate values programmable outside the root context.

Comparison of mechanisms, not mutually exclusive product categories. Zhang et al. (2026).

Choose an approach for the evidence you need

Choose a starting point from the question and the source collection.

Evidence need	Possible starting point
All sources fit in one call	Whole-document input; cache repeats
Chunks omit needed context	Parent sections or contextual prefix
Exact lookup across many sources	Hybrid retrieval and reranking
Follow entity relationships	Graph-based retrieval
First search leaves a missing fact	Agentic RAG
Research a broad topic for a report	Research, source notes, then outline
Inspect a large archive using code	Recursive source inspection

Compare evidence coverage, answer support, time, and cost. Approaches can be combined.

Li et al. (2025) Edge et al. (2024; v2 2025) Anthropic (2024) Chan et al. (2025) Zhang et al. (2025; v3 2026)

Find the failed step before changing the workflow

Locate the failed step before changing the model or adding more searches.

Observed failure	Inspect or change
A chunk lost its heading or uses an old rule	Text preparation and source version
A needed chunk never appeared	Retrieval results and search query
A found chunk never reached the model	The exact generator input
A supplied source does not support the answer	Each claim and its cited source text
Research still leaves important questions open	Source notebook, outline, and search limits

Retest on the same questions and sources. Compare answer quality, support, time, and cost.

LA source snapshot: 21 Oct 2026. Karpukhin et al. (2020) Lewis et al. (2020) Gao et al. (2023)

Choose the simplest adequate workflow

One retrieval pass	precise source question
Dependent searches	next fact depends on a result
Full-context answering	the collection fits in context

Preserve sources, check each claim, and stop with explicit gaps.

Lewis et al. (2020) Asai et al. (2024) Gao et al. (2023) Chan et al. (2025)

References: retrieval, generation, and source checking

Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering

Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Gao et al. (2023). Enabling Large Language Models to Generate Text with Citations

References: adaptive retrieval

Asai et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection

Self-RAG: ICLR 2024. Figures use the pinned 2023 preprint.

Further reading: design options and current research

Edge et al. (2024; v2 2025). From Local to Global: A Graph RAG Approach to Query-Focused Summarization

Sarthi et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

Anthropic (2024). Introducing Contextual Retrieval

Chan et al. (2025). Don’t Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks

Zhang et al. (2025; v3 2026). Recursive Language Models

Li et al. (2025). LaRA: Benchmarking Retrieval-Augmented Generation and Long-Context LLMs: No Silver Bullet for LC or RAG Routing

Microsoft (accessed 2026). GraphRAG local and global search documentation